Meu Aprendizado de Máquinas

Ronald A. Mendonça

2026

Sumário

Prefácio **5**

Agradecimentos **6**

Introdução **7**

I Aprendizado de Máquina Clássico **8**

1 Conceitos Fundamentais: Dados e Aprendizado **9**
1.1 Dados 9
1.2 Importância da Compreensão dos Dados 10
1.3 Tipos de Aprendizado 10
1.3.1 Aprendizado Supervisionado (Supervised Learning) 11
1.3.2 Aprendizado não Supervisionado (Unsupervised Learning) 11
1.3.3 Aprendizado por Reforço (Reinforcement Learning) 12
1.3.4 Aprendizado Semissupervisionado (Semi-Supervised Learning) . . . 12

2 Estatística Essencial para Machine Learning **14**
2.1 Estatística Descritiva 14
2.1.1 Calculando a Média e o Desvio Padrão de Temperaturas Diárias . . 15
2.1.2 Aplicando Python 16
2.2 Distribuições de Dados 17
2.2.1 Distribuição Normal e Sua Relevância 18
2.2.2 Exemplo Prático: Verificando Normalidade em Dados de Altura . . 18
2.3 Correlação de Pearson 19
2.3.1 Exemplo Prático 20
2.3.2 Resultado 21
2.4 Testes de Hipóteses 22
2.4.1 Conceito de p-valor e Teste t 23
2.4.2 Exemplo Prático: Comparando Médias de Dois Grupos 23
2.5 Normalização e Escalonamento 23
2.5.1 Por que Normalizar Dados para Machine Learning 23

2.5.2 Exemplo Prático: Escalonando Características para um Modelo . . 24
2.6 Outliers em Modelos de Regressão 24
2.6.1 Exemplo Prático: Detecção de Outliers em Preços de Casas 25
2.6.2 Interpretação dos Resultados 28

3 Regressão: Previsão de Valores Contínuos 29
3.1 O que é Regressão? 29
3.2 Regressão Linear 29
3.3 Exemplo Prático: Prever o Preço de Casas 30
3.3.1 Cálculo Manual Simplificado 31
3.3.2 Calculando com Python 32
3.3.3 Interpretação 32
3.4 Notação Algébrica e Vetorial 32
3.4.1 Convenções 33
3.4.2 Regressão Linear Múltipla 33
3.4.3 Vetores e Produto Escalar 34
3.4.4 A Matriz de Dados 34
3.4.5 A Função de Custo em Forma Vetorial 35
3.4.6 A Equação Normal 36
3.4.7 Equação Normal em Python 37
3.4.8 Interpretando os Pesos 38
3.4.9 Limitações da Solução Fechada 39
3.5 Árvore de Decisão 40
3.5.1 Árvores de Decisão para Regressão 40
3.6 Avaliação de Modelos de Regressão 44

4 Classificação: Previsão de Categorias 45
4.1 O que é Classificação? 45
4.2 Modelos Básicos de Classificação 45
4.2.1 Regressão Logística 45
4.2.2 K-Nearest Neighbors (KNN) 51
4.3 Considerações Finais 52

5 Clustering: Agrupamento de Dados 53
5.1 O que é Aprendizado Não Supervisionado e Clustering? 53
5.2 Algoritmo K-Means 53
5.3 Exemplo Prático: Segmentar Clientes por Compras 54
5.4 Visualização dos Clusters 55
5.5 Interpretação e Limitações 57

6 Avaliação e Melhoria de Modelos 58
6.1 Overfitting e Underfitting: Definições e Causas 58
6.2 Divisão Treino/Teste e Validação Cruzada 58

6.3 Exemplo Prático: Avaliar um Modelo de Regressão . . . 59
6.4 Métricas de Avaliação . . . 60
6.5 Matriz de Confusão . . . 61
6.5.1 Exemplo Prático: Detecção de Fraudes em Cartões de Crédito . . . 61
6.5.2 Interpretação dos Resultados . . . 64
6.6 Interpretação e Próximos Passos . . . 65

7 Aplicando o Aprendizado em Dados Reais 66
7.1 Ferramentas e Preparo . . . 66
7.1.1 Python e sua Versão Atual . . . 66
7.1.2 Google Colab e Jupyter Notebook . . . 66
7.1.3 Scikit-learn . . . 67
7.2 Classificando Notícias com Aprendizado de Máquina . . . 67
7.2.1 Conectando ao Google Drive . . . 67
7.2.2 Download do arquivo para testes . . . 67
7.2.3 Carregando o Dataset . . . 68
7.2.4 Analisando o Dataset . . . 68
7.2.5 Preparando os Dados Classificados . . . 69
7.2.6 Transformando Texto em Números . . . 70
7.2.7 Treinando e Testando o Modelo . . . 70
7.2.8 Hora do Treinamento . . . 71
7.2.9 Testando com Novas Notícias . . . 72
7.2.10 (Opcional) Classificando Todas as Notícias . . . 72
7.3 Síntese Conceitual das Técnicas Utilizadas . . . 73
7.4 Para ir mais além . . . 74

II Deep Learning 75

8 Do Neurônio às Redes Neurais 76

9 Como uma Rede Aprende 77

10 Treinando Redes Neurais na Prática 78

11 Recursos e Considerações Finais 79
11.1 Recursos Recomendados . . . 79
11.2 Considerações Finais . . . 79

Prefácio

Em 2019, concluí minha graduação em Gestão da Tecnologia da Informação e, em meados de 2020, iniciei uma pós-graduação em Data Science e Inteligência Artificial. Desde os tempos de escola, tenho o hábito de fazer anotações sobre aquilo que estudo. Naquela época, eram registros sobre frações em matemática ou análise morfológica nas intermináveis aulas de português. Anos depois, esse hábito continuou presente durante minha formação acadêmica e em meus estudos por conta própria.

Com o tempo, minhas anotações passaram a ocupar diferentes lugares, formatos e arquivos. Eram exercícios, explicações, dúvidas, exemplos de código e experimentos com novas tecnologias. Parte desse material surgiu durante os estudos formais; outra parte, da curiosidade de explorar conceitos e desenvolver projetos pessoais.

Este livro começou como um pequeno projeto experimental, reunindo alguns desses projetos e registros. À medida que o material cresceu, percebi que poderia organizá-lo de maneira mais estruturada, transformando anotações dispersas em uma obra que pudesse consultar, revisar e ampliar ao longo do tempo.

O resultado é uma compilação de conhecimentos que venho adquirindo ao longo dos anos, especialmente nas áreas de Machine Learning, Deep Learning e Visão Computacional. Não se trata de um trabalho definitivo, tampouco de uma tentativa de esgotar os assuntos abordados. É, antes de tudo, o registro de uma trajetória de aprendizado, construída por meio de estudos, exercícios, erros, experimentos e descobertas.

Escrevo este livro, inicialmente, para mim mesmo: como uma forma de consolidar o que aprendi, identificar lacunas no meu conhecimento e estabelecer conexões entre conceitos que, muitas vezes, são estudados separadamente. Organizar uma explicação, desenvolver um exemplo ou reproduzir um procedimento também são formas de verificar se realmente compreendemos aquilo que estamos estudando.

Ainda assim, espero que este material possa ser útil a outras pessoas. Se você está iniciando seus estudos em aprendizado de máquina, aprofundando seus conhecimentos em inteligência artificial ou começando a explorar a visão computacional, talvez encontre nestas páginas uma explicação, um exemplo ou uma referência que facilite sua própria jornada.

Assim como eu, você provavelmente terá dúvidas, cometerá erros e precisará revisitar conceitos várias vezes. Isso faz parte do processo. Este livro é o resultado desse mesmo percurso e continuará evoluindo à medida que eu aprender coisas novas.

Mais do que apresentar respostas definitivas, espero que ele sirva como um registro de aprendizado e um ponto de partida para novas perguntas, experimentos e descobertas.

Agradecimentos

...

Introdução

Machine Learning (ML), ou aprendizado de máquina, é uma área da inteligência artificial que permite que sistemas computacionais identifiquem padrões e façam previsões a partir de dados, sem a necessidade de programação explícita para cada tarefa. Diferente de softwares tradicionais, onde regras são definidas manualmente, em ML os algoritmos aprendem diretamente com exemplos, ajustando-se para melhorar seu desempenho ao longo do tempo. Essa capacidade transformou campos como medicina, finanças e tecnologia, tornando-se uma das competências mais valiosas do século XXI.

A relevância do Machine Learning está em sua aplicabilidade prática. Empresas utilizam ML para prever vendas, detectar fraudes e personalizar recomendações, enquanto cientistas o aplicam para analisar dados complexos, como sequências genéticas ou padrões climáticos. Para o iniciante, entender os fundamentos de ML abre portas para uma carreira em ciência de dados ou simplesmente para compreender melhor o mundo movido a dados em que vivemos.

Neste livro reúno os conceitos fundamentais para iniciantes que desejam dar os primeiros passos em Machine Learning, sem experiência prévia na área. Para acompanhar, é recomendável um conhecimento básico de matemática (estatística e equações lineares) e familiaridade mínima com programação, preferencialmente em Python, embora os conceitos sejam explicados de forma acessível. Meu objetivo é desmistificar o ML, oferecendo uma base teórica para quem quer começar a construir modelos simples e entender seus resultados.

Ao longo das próximas páginas, você será introduzido aos conceitos essenciais de Machine Learning, desde a manipulação de dados até a criação de modelos de regressão, classificação e clustering. Cada capítulo combina teoria com exemplos práticos, incluindo trechos de código em Python para ilustrar a aplicação dos métodos discutidos. Ao final, você terá conteúdo suficiente para construir um modelo de machine learning simples e entender os princípios que o sustentam.

O livro está organizado em três partes. A Parte I cobre o aprendizado de máquina clássico, dos dados à avaliação de modelos, e termina com um projeto em dados reais. A Parte II trata de Deep Learning, e a Parte III, de Visão Computacional.

Parte I

Aprendizado de Máquina Clássico

Capítulo 1

Conceitos Fundamentais: Dados e Aprendizado

1.1 Dados

Em um dia de trabalho, você se depara com o desafio de extrair respostas a partir de um conjunto de informações que, à primeira vista, parecem desconexas. Esses dados podem estar estruturados ou não e assumir diferentes formatos, como números, textos, imagens ou qualquer outro tipo de informação. Talvez, para você, esse conjunto não faça o menor sentido, mas acredite, a máquina é capaz de aprender e encontrar padrões de forma extremamente eficaz. Os dados são a base de qualquer modelo de Machine Learning. Eles precisam ser coletados, organizados e analisados para que possamos extrair insights valiosos e transformar informações brutas em conhecimento útil.

- **Dados Qualitativos (ou Categóricos):** Representam características ou atributos que não podem ser quantificados numericamente.
  - **Nominais:** Não possuem ordem natural. Ex: cores (vermelho, azul, verde), tipos de frutas.
  - **Ordinais:** Possuem uma ordem ou hierarquia. Ex: níveis de satisfação (ruim, regular, bom, ótimo), ranking de qualidade (A, B, C).
- **Dados Quantitativos (ou Numéricos):** Representam valores numéricos e podem ser medidos.
  - **Discretos:** Assumem valores contáveis, geralmente inteiros. Ex: número de filhos, número de carros em um estacionamento.
  - **Contínuos:** Podem assumir qualquer valor dentro de um intervalo. Ex: altura, peso, temperatura.

1.2 Importância da Compreensão dos Dados

A compreensão dos diferentes tipos de dados constitui um pilar fundamental para o sucesso de qualquer projeto de aprendizado de máquina, pois influencia diretamente tanto a escolha dos algoritmos mais apropriados quanto as estratégias de preparação e transformação das informações. Dados quantitativos, como preços de ações ou temperaturas, podem ser diretamente integrados a modelos matemáticos e estatísticos, aproveitando sua natureza estruturada e mensurável. Por outro lado, dados qualitativos ou textuais, como opiniões expressas em notícias, comentários em redes sociais ou categorias não numéricas, demandam etapas adicionais de pré-processamento para serem convertidos em formatos que os algoritmos possam interpretar. Essas etapas podem incluir tokenização, remoção de palavras irrelevantes (*stopwords*) ou a aplicação de técnicas como o TF-IDF (*Term Frequency-Inverse Document Frequency*), que transformam textos em representações numéricas significativas.

A relevância dessa distinção pode ser ilustrada por uma situação prática. Imagine uma empresa de mídia que, em 2018, decidiu analisar o sentimento público em relação a uma nova política econômica com base em milhares de artigos de notícias coletados online. Inicialmente, a equipe tentou aplicar um modelo de regressão diretamente sobre os textos brutos, sem qualquer tratamento, assumindo que os algoritmos dariam conta do recado. O resultado foi desastroso: previsões sem sentido e um modelo incapaz de capturar padrões, pois os dados textuais não haviam sido adequadamente preparados. Foi somente após a intervenção de um cientista de dados, que identificou a necessidade de categorizar os dados como qualitativos e aplicar técnicas de vetorização, que o projeto tomou forma. Com o pré-processamento correto, o modelo passou a identificar com precisão sentimentos positivos e negativos, permitindo à empresa ajustar sua estratégia editorial em tempo real. Essa história sublinha um princípio básico: sem entender a natureza dos dados, mesmo os algoritmos mais avançados podem falhar.

Nos capítulos seguintes, essa distinção entre tipos de dados será explorada em maior profundidade, com foco em exemplos práticos e detalhados. Abordaremos como transformar dados textuais em representações numéricas úteis, preparar conjuntos quantitativos para análises preditivas e integrá-los em tarefas reais de aprendizado de máquina, como a classificação de sentimentos em notícias. Esse conhecimento não apenas facilita a seleção de métodos adequados, mas também garante que os resultados obtidos sejam confiáveis e aplicáveis, estabelecendo uma base sólida para projetos mais complexos no campo da ciência de dados.

1.3 Tipos de Aprendizado

Os tipos de aprendizado de máquina podem ser classificados em quatro categorias principais: supervisionado, não supervisionado, por reforço e semissupervisionado.

1.3.1 Aprendizado Supervisionado (Supervised Learning)

O aprendizado supervisionado é o tipo mais comum e intuitivo de ML, usado quando temos um conjunto de dados rotulados — ou seja, cada exemplo de entrada vem com uma resposta correta (o "supervisor"). O objetivo é treinar um modelo para mapear entradas (*features*) para saídas (*labels*), aprendendo uma função que generalize bem para dados novos. Ele se divide em duas categorias principais: **classificação**, quando a saída é uma categoria (ex.: "spam"ou "não spam"), e **regressão**, quando a saída é um valor contínuo (ex.: preço de uma casa).
Imagine um banco que quer prever a possibilidade de um determinado cliente pagar um empréstimo tomado. O dataset tem *features* como renda anual, histórico de crédito e idade, e um rótulo binário: "pago"(1) ou "não pago"(0). Um modelo de classificação (ex.: regressão logística) aprende a partir desses dados históricos e, ao receber um novo cliente, prevê a probabilidade de pagamento. Se o modelo acerta 85% das vezes no teste, o banco pode usá-lo para aprovar ou negar empréstimos com mais confiança.
A supervisão garante precisão alta quando os dados são bem rotulados, mas exige um grande esforço para coletar e anotar esses rótulos — pense em milhares de imagens marcadas manualmente ou registros financeiros validados. Dados enviesados ou rótulos errados também podem arruinar o modelo.

- Previsão de preços de imóveis
- Diagnóstico médico
- Classificação de e-mails como spam ou não spam

1.3.2 Aprendizado não Supervisionado (Unsupervised Learning)

O aprendizado não supervisionado deverá ser usado quando não temos rótulos — apenas dados brutos e a missão de encontrar padrões ou estruturas escondidas. Aqui, o modelo não é "ensinado"com respostas certas; ele explora os dados por conta própria, agrupando itens semelhantes ou reduzindo a complexidade. As duas tarefas principais são **clustering** (agrupamento, ex.: segmentar clientes) e **redução de dimensionalidade** (ex.: comprimir dados mantendo informações chave).
O processo consiste numa investigação de pistas: algoritmos como K-Means ou PCA (Análise de Componentes Principais) analisam as *features*, calculam distâncias ou variâncias, e organizam os dados em grupos ou representações simplificadas. Não há um "certo"ou "errado"explícito — a qualidade depende do quão útil é o padrão descoberto.
Uma loja online tem dados de compras dos clientes (ex.: itens comprados, frequência, valor gasto), mas sem categorias definidas. Usando K-Means, o modelo agrupa os clientes em clusters: "compradores casuais", "fanáticos por tecnologia", "caçadores de promoções". A loja usa isso pra personalizar campanhas de marketing, sem precisar rotular cada cliente manualmente.
A ausência de rótulos economiza tempo e esforço, abrindo portas pra explorar dados novos

ou caóticos. Mas avaliar o resultado é mais subjetivo — como saber se os clusters fazem sentido? — e exige ajustes manuais (ex.: escolher o número de grupos no K-Means).

- Segmentação de clientes
- Compressão de dados
- Detecção de anomalias

1.3.3 Aprendizado por Reforço (Reinforcement Learning)

O aprendizado por reforço é o tipo mais "jogo"do ML, inspirado em como humanos aprendem por tentativa e erro. Aqui, não há um dataset fixo com entradas e saídas; em vez disso, um agente interage com um ambiente, tomando ações e recebendo recompensas (ou punições) com base nos resultados. O objetivo é maximizar a recompensa total ao longo do tempo, aprendendo uma política — um plano de ação — que guia as decisões.
Funciona assim: o agente observa o estado do ambiente (ex.: posição num jogo), escolhe uma ação (ex.: mover pra direita), recebe uma recompensa (ex.: +10 pontos) e ajusta sua estratégia usando algoritmos como Q-Learning ou Deep Reinforcement Learning (DRL). É um ciclo contínuo de exploração e aprendizado.
Pense num robô aspirador navegando numa casa. O ambiente é o mapa da casa, as ações são "virar à esquerda", "avançar", "parar", e a recompensa vem por limpar sujeira (+5), enquanto bater na parede gera uma punição (-2). Após várias tentativas, o robô aprende a trajetória ideal pra limpar tudo sem travar. A DeepMind usou essa abordagem pra treinar agentes que jogam videogames do Atari e, com o AlphaGo, o jogo de tabuleiro Go. É perfeito pra problemas dinâmicos e interativos (ex.: robótica, jogos), mas exige tempo e poder computacional proporcional para treinar — o agente pode demorar pra aperfeiçoar o treinamento. Definir as recompensas também é um desafio: se mal calibradas, o modelo pode aprender coisas inúteis.

- Jogos (Atari, xadrez, Go)
- Controle de robôs

1.3.4 Aprendizado Semissupervisionado (Semi-Supervised Learning)

O aprendizado semissupervisionado é o meio-termo entre o supervisionado e o não supervisionado, usado quando temos poucos dados rotulados e muitos sem rótulos. A ideia é combinar o poder dos rótulos pra guiar o aprendizado com a quantidade dos dados não rotulados pra enriquecer o modelo. Ele mistura técnicas dos dois mundos: primeiro, usa os dados rotulados pra treinar um modelo básico, depois "propaga"esse conhecimento pros dados sem rótulos, refinando as previsões.
O processo pode envolver **pseudo-rotulação** (o modelo prevê rótulos pros dados não

rotulados e os inclui no treino) ou **co-treinamento** (dois modelos colaboram, cada um rotulando dados pro outro). É uma solução prática pra cenários reais, onde rotular tudo é caro ou impossível.
Uma empresa de saúde tem 100 imagens médicas rotuladas como "normal"ou "tumor"e 10.000 imagens sem rótulos. Um modelo supervisionado treina nos 100 exemplos, prevê rótulos pras 10.000, e depois retreina com o conjunto expandido. Isso melhora a detecção de tumores em exames futuros, aproveitando os dados não rotulados que seriam ignorados num aprendizado puramente supervisionado.
Economiza esforço de rotulação e aumenta a robustez com mais dados, mas depende da qualidade dos rótulos iniciais — se tiverem erros, a propagação pode amplificar o problema. Também exige algoritmos mais complexos pra balancear as duas fontes de informação.

- Análise de documentos jurídicos (grande volume de textos sem rótulo).
- Sistemas de recomendação (mistura de interações explícitas e implícitas).

Capítulo 2

Estatística Essencial para Machine Learning

2.1 Estatística Descritiva

A estatística descritiva é uma parte fundamental da análise de dados, pois fornece uma visão geral das características principais de um conjunto de dados. Ela envolve o resumo e a descrição dos dados por meio de medidas numéricas, gráficos e tabelas. As principais medidas incluem:

- **Média:** A média aritmética é a estimativa de localização mais básica. A média é a soma de todos os valores dividida pelo número total de valores.
- **Mediana:** É o valor que separa os dados em duas metades, que ocupa a posição central em um conjunto de dados ordenado (organizado de forma crescente ou decrescente). Se houver um número ímpar de dados, ela será o valor do meio. Se houver um número par de dados, será a média aritmética dos dois valores centrais.
- **Moda:** É o valor que aparece com mais frequência em um conjunto de dados. Suponha o conjunto `[2,4,5,5,6,7,5]`, o valor de moda será o `5`. O mesmo acontece no conjunto `[A, B, C, A, D, A, E]`, o valor de moda será o `A`.
- **Variância:** Uma medida da dispersão dos dados em relação à média. A variância é calculada como a média dos quadrados das diferenças entre cada valor e a média.
- **Desvio Padrão:** Raiz quadrada da variância, expressa na mesma unidade dos dados. Quanto maior o desvio padrão, mais espalhados estão os dados.
- **Correlação:** Uma medida que indica a força e a direção da relação entre duas variáveis. A correlação varia de -1 a 1, onde -1 indica uma correlação negativa perfeita, 0 indica nenhuma correlação e 1 indica uma correlação positiva perfeita.
- **Outlier:** Valor extremo, muito diferente da maioria dos dados.

2.1.1 Calculando a Média e o Desvio Padrão de Temperaturas Diárias

Suponha que registramos as temperaturas máximas diárias (°C) de uma semana em uma cidade. Os dados coletados são: 20, 22, 19, 23, 21, 20, 24. Vamos calcular a média e o desvio padrão passo a passo.

Cálculo da Média

A média ($\bar{x}$) é obtida somando todos os valores e dividindo pelo número de observações (n). Matematicamente:

$$\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n} \tag{2.1}$$

Para os dados fornecidos:

- Soma dos valores: $20 + 22 + 19 + 23 + 21 + 20 + 24 = 149$,
- Número de observações: $n = 7$,
- Média: $\bar{x} = \frac{149}{7} \approx 21{,}29$.

Portanto, a temperatura média diária é aproximadamente 21,29°C.

Cálculo do Desvio Padrão

O desvio padrão (σ) mede a variabilidade dos dados em relação à média. Para uma população, é dado por:

$$\sigma = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n}} \tag{2.2}$$

Passo a passo:

1. Calcule as diferenças entre cada valor e a média ($x_i - \bar{x}$):
   - $20 - 21{,}29 = -1{,}29$,
   - $22 - 21{,}29 = 0{,}71$,
   - $19 - 21{,}29 = -2{,}29$,
   - $23 - 21{,}29 = 1{,}71$,
   - $21 - 21{,}29 = -0{,}29$,
   - $20 - 21{,}29 = -1{,}29$,
   - $24 - 21{,}29 = 2{,}71$.
2. Eleve cada diferença ao quadrado:
   - $(-1{,}29)^2 = 1{,}6641$,

- $0{,}71^2 = 0{,}5041$,
- $(-2{,}29)^2 = 5{,}2441$,
- $1{,}71^2 = 2{,}9241$,
- $(-0{,}29)^2 = 0{,}0841$,
- $(-1{,}29)^2 = 1{,}6641$,
- $2{,}71^2 = 7{,}3441$.

3. Some os quadrados: $1{,}6641 + 0{,}5041 + 5{,}2441 + 2{,}9241 + 0{,}0841 + 1{,}6641 + 7{,}3441 = 19{,}4288$,

4. Divida pelo número de observações: $\frac{19{,}4288}{7} \approx 2{,}7755$,

5. Tire a raiz quadrada: $\sigma = \sqrt{2{,}7755} \approx 1{,}67$.

Assim, o desvio padrão das temperaturas é aproximadamente 1,67°C.

2.1.2 Aplicando Python

Os mesmos cálculos podem ser realizados de forma eficiente usando Python com a biblioteca NumPy. O código a seguir mostra como:

```python
# Importando a biblioteca NumPy
import numpy as np

# Dados das temperaturas diárias
temperaturas = [20, 22, 19, 23, 21, 20, 24]

# Calcula a média
media = np.mean(temperaturas)
print(f"Média das temperaturas: {media:.2f}°C")

# Calcula o desvio padrão
desvio_padrao = np.std(temperaturas)
print(f"Desvio padrão das temperaturas: {desvio_padrao:.2f}°C")
```

Executando o código, obtemos uma média de 21,29°C e um desvio padrão de 1,67°C, os mesmos valores do cálculo manual. Por padrão, `np.std` calcula o desvio padrão populacional (divisão por n), o mesmo da fórmula acima; para o desvio amostral (divisão por $n-1$), use `np.std(temperaturas, ddof=1)`.

Visualização com Curva Normal

A distribuição das temperaturas pode ser visualizada em uma curva normal, que mostra como os dados se distribuem em torno da média. A Figura 2.1 apresenta o histograma das temperaturas e a curva normal correspondente, gerada com Python e as bibliotecas Matplotlib e SciPy.

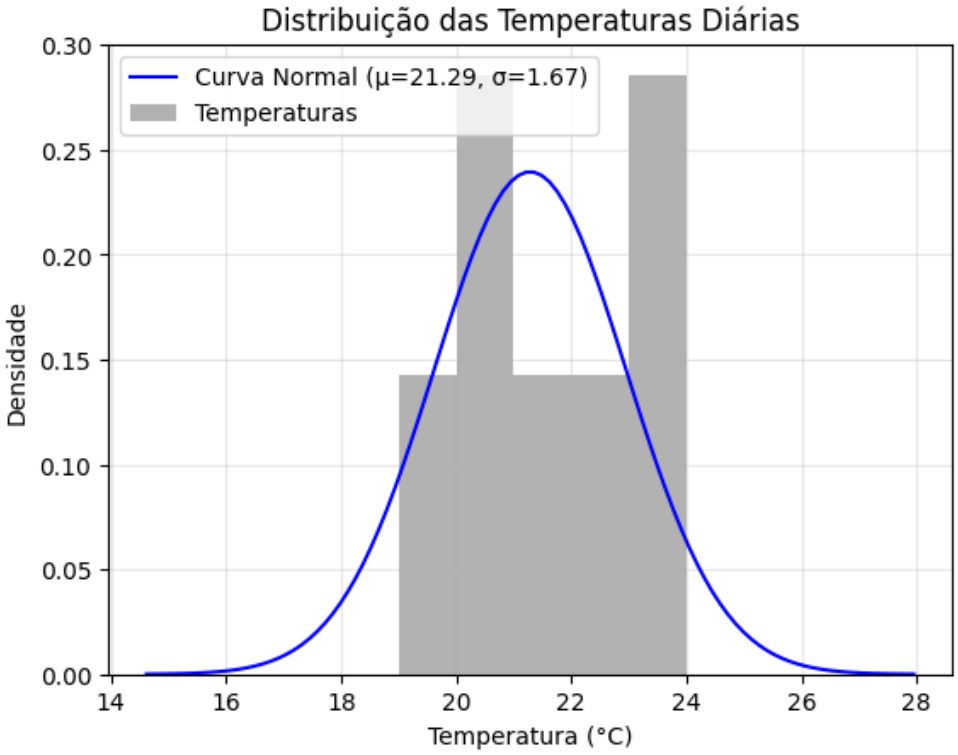


Figura 2.1: Distribuição das temperaturas diárias com curva normal (média = 21,29°C, desvio padrão = 1,67°C).

Interpretação

A média de 21,29°C e o desvio padrão de 1,67°C indicam que as temperaturas variam moderadamente em torno de um valor central. A curva normal na Figura 2.1 ilustra essa distribuição, sendo útil em Machine Learning para entender a variabilidade dos dados antes de aplicar modelos preditivos.

2.2 Distribuições de Dados

Como hoje temos um grande volume de dados com diferentes níveis de qualidade e relevância, a amostragem é essencial, pois permite lidar com esses dados de maneira eficiente e reduzir possíveis vieses. Mesmo em iniciativas que envolvem big data, os modelos

preditivos costumam ser inicialmente construídos e testados com amostras (divisão entre teste e treinamento). Além disso, amostras são frequentemente utilizadas em diversos experimentos, como testes de preços ou variações em páginas da web.

2.2.1 Distribuição Normal e Sua Relevância

A distribuição normal, ou gaussiana, é uma curva simétrica em forma de sino, definida por sua média (μ) e desvio padrão (σ). Aproximadamente 68% dos dados estão a 1 desvio padrão da média, 95% a 2 desvios, e 99,7% a 3 desvios. A normalidade é uma premissa de alguns métodos específicos, e não um pré-requisito geral de *Machine Learning*. O Gaussian Naive Bayes e a Análise Discriminante Linear assumem *features* normais dentro de cada classe; na regressão linear, a suposição recai sobre os resíduos e só importa para a inferência clássica (intervalos de confiança e testes sobre os coeficientes). Árvores de decisão, Random Forest, KNN, SVM e redes neurais não exigem que as variáveis sigam distribuição normal. Ainda assim, variáveis muito assimétricas podem se beneficiar de transformações (ex.: logarítmica).

2.2.2 Exemplo Prático: Verificando Normalidade em Dados de Altura

Geramos um conjunto fictício de alturas (média 170 cm, desvio 10 cm) e visualizamos sua distribuição:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

np.random.seed(42)  # semente fixa para reproduzir o resultado
alturas = np.random.normal(170, 10, 100)
plt.hist(alturas, bins=10, density=True, alpha=0.6, color='b',
↪  label='Histograma')
xmin, xmax = plt.xlim()
x = np.linspace(xmin, xmax, 100)
p = norm.pdf(x, 170, 10)
plt.plot(x, p, 'k', linewidth=2, label='Curva Normal')
plt.title("Distribuição de Alturas")
plt.xlabel("Altura (cm)")
plt.ylabel("Densidade")
plt.legend()
plt.show()

from scipy.stats import shapiro
```

```
estatistica, p_valor = shapiro(alturas)
print(f"Teste Shapiro - Estatística: {estatistica:.3f}, p-valor:
↪  {p_valor:.3f}")
```

Resultados: O histograma segue aproximadamente a curva normal (Figura 2.2). No teste de Shapiro-Wilk, o p-valor foi de 0,655, acima de 0,05: não há evidência para rejeitar a hipótese de normalidade.

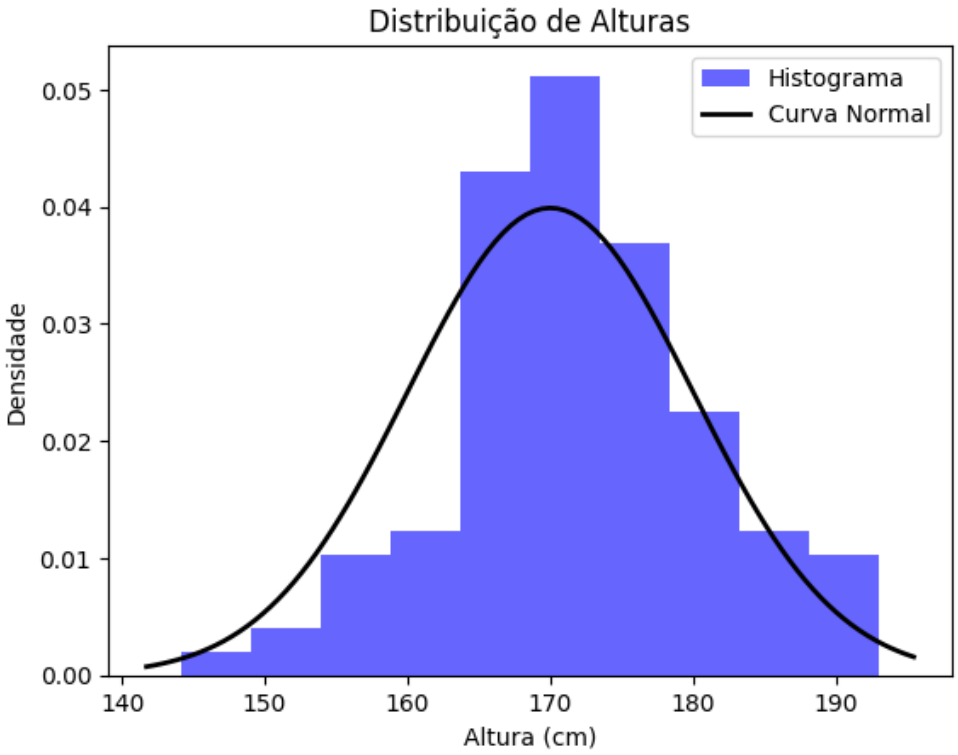


Figura 2.2: Distribuição de alturas com curva normal.

Interpretação: Dados aproximadamente normais atendem aos métodos que dependem dessa suposição. Se o p-valor fosse menor que 0,05, valeria avaliar uma transformação (ex.: log) antes de aplicar testes paramétricos ou modelos que assumem normalidade.

2.3 Correlação de Pearson

A análise exploratória dos dados, na maioria dos projetos de modelagem, seja para pesquisa ou para ciência de dados, envolve o estudo da correlação entre variáveis, e entre variáveis e uma variável-alvo. A correlação de Pearson r mede a força e direção da relação linear entre duas variáveis X e Y. A fórmula é dada por:

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \cdot \sum(y_i - \bar{y})^2}}$$

2.3.1 Exemplo Prático

Considere os seguintes dados abaixo:

Pessoa	**Horas de Estudo (X)**	**Nota na Prova (Y)**
João	2	60
Ana	4	70
Bruno	6	80
Carla	8	90
Lucas	10	100

1. Cálculo das Médias

Para (x) somamos as horas de estudo de todas as pessoas e dividimos pelo total de pessoas. E para (y) faremos o mesmo, soma-se as notas nas provas e dividimos pela quantidade de notas.

$$\bar{x} = \frac{2+4+6+8+10}{5} = 6 \quad \text{e} \quad \bar{y} = \frac{60+70+80+90+100}{5} = 80$$

2. Tabela de Cálculo

Observando a seguinte tabela de cálculo, poderemos, em seguida, substituir cada um dos elementos na fórmula de correlação.

x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$	$(y_i - \bar{y})^2$
2	60	-4	-20	80	16	400
4	70	-2	-10	20	4	100
6	80	0	0	0	0	0
8	90	2	10	20	4	100
10	100	4	20	80	16	400

3. Substituindo na Fórmula

Ao substituir os elementos da fórmula, poderemos, enfim, chegar a uma conclusão a respeito da relação das notas das provas e a quantidade de horas estudadas por cada pessoa.

$$r = \frac{80+20+0+20+80}{\sqrt{(16+4+0+4+16)(400+100+0+100+400)}} =$$

$$r = \frac{200}{\sqrt{40 \cdot 1000}} = \frac{200}{\sqrt{40000}} = \frac{200}{200} = 1$$

2.3.2 Resultado

A correlação entre horas de estudo e nota na prova é $r = 1$, indicando uma **correlação positiva perfeita**. Ou seja, quanto mais a pessoa estuda, maior tende a ser sua nota.

Aplicando Python com mesmo exemplo

Utilizamos a correlação de Pearson, só que dessa vez aplicando código em Python, que gerou a Figura 2.3, ilustrando essa relação, além de fazer o cálculo.

Código utilizado:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import pearsonr

# Dados fictícios: Horas estudadas e notas obtidas
horas_estudo = np.array([2, 4, 6, 8, 10])
notas = np.array([60, 70, 80, 90, 100])

# Cálculo da correlação de Pearson
correlacao, p_valor = pearsonr(horas_estudo, notas)

# Exibir os resultados
print(f"Correlação de Pearson: {correlacao:.2f}")
print(f"Valor-p: {p_valor:.4f}")

# Gerar gráfico de dispersão
plt.figure(figsize=(8, 5))
plt.scatter(horas_estudo, notas, color='blue', label="Dados")
plt.plot(horas_estudo, np.poly1d(np.polyfit(horas_estudo, notas,
↪  1))(horas_estudo), color="red", linestyle="--", label="Tendência")
plt.title('Correlação entre Horas Estudadas e Notas Obtidas')
plt.xlabel('Horas Estudadas')
plt.ylabel('Nota Obtida')
plt.legend()
plt.grid(False)
plt.tight_layout()
plt.show()
```

```
Saída: Correlação de Pearson: 1.00
Valor-p: 0.0000
```

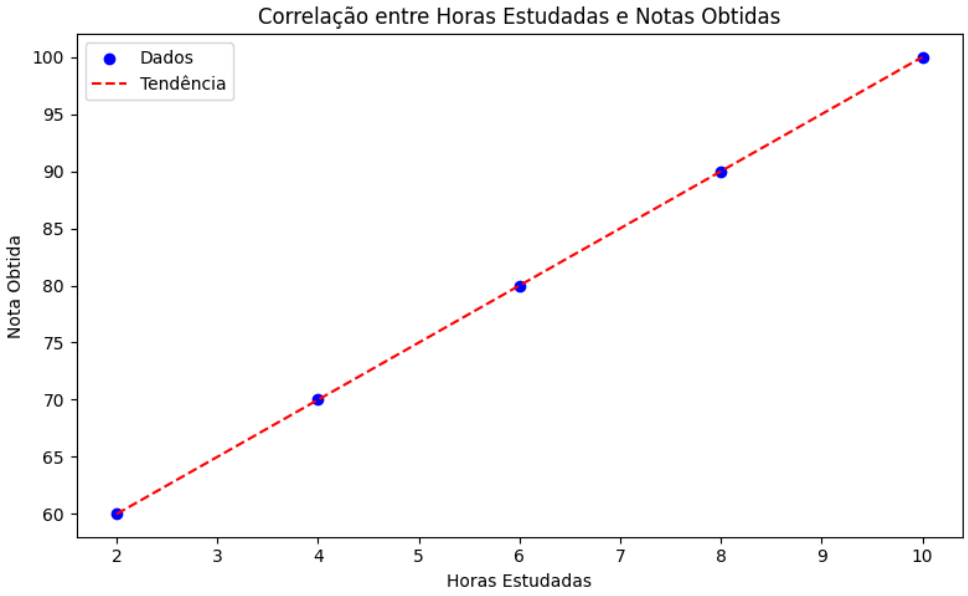


Figura 2.3: Correlação entre horas estudadas e notas obtidas.

O coeficiente de correlação de Pearson encontrado foi de exatamente $r = 1$, indicando uma **correlação positiva perfeita** assim como feito manualmente no exemplo anterior. Isso significa que, à medida que o número de horas estudadas aumenta, as notas dos alunos tendem a melhorar.

2.4 Testes de Hipóteses

Testes de hipóteses avaliam se diferenças ou relações nos dados são estatisticamente significativas, sendo capaz de orientar decisões. Imagine que todo dia você sai de casa e tem duas opções de caminho para ir ao trabalho: a Rua A e a Rua B. Você quer descobrir qual delas é mais rápida, mas há muitas variáveis envolvidas: semáforos, buracos, quantidade de carros, etc. Então você decide fazer um experimento: por 10 dias, metade das vezes você vai pela Rua A e, nas outras, pela Rua B. No final de cada viagem, você anota o tempo que levou. Depois de juntar os dados, você percebe que a Rua B parece ser mais rápida, mas será que é realmente mais eficiente ou foi só sorte? É aí que entra o teste A/B, um tipo de teste de hipótese. Com ele, você consegue verificar estatisticamente se a diferença entre os tempos das ruas é significativa, ou se as variações foram apenas por acaso (como um dia sem trânsito ou um semáforo quebrado). Em machine learning, A/B testing funciona do mesmo jeito. Suponha que você tenha dois modelos de recomendação: o modelo A (antigo) e o modelo B (novo). Você mostra os dois para diferentes usuários e mede qual gera mais cliques ou vendas. No fim, você faz um teste estatístico (como no exemplo das ruas) pra saber qual modelo realmente performa melhor, sem confiar só na sorte. A lógica é simples: testar, medir e decidir com base em dados reais.

2.4.1 Conceito de p-valor e Teste t

O teste t compara médias de dois grupos, testando a hipótese nula (H_0) de que não há diferença significativa. O p-valor indica a probabilidade de observar uma diferença tão grande quanto a encontrada (ou maior) caso H_0 seja verdadeira; por convenção, um p-valor menor que 0,05 leva a rejeitar H_0, sugerindo diferença real. Em ML, isso ajuda a determinar se uma *feature* impacta a variável alvo.

2.4.2 Exemplo Prático: Comparando Médias de Dois Grupos

Comparamos temperaturas de duas cidades:

```
from scipy.stats import ttest_ind

temp_cidade1 = [22, 23, 22, 25, 19]
temp_cidade2 = [20, 21, 19, 22, 18]
estatistica, p_valor = ttest_ind(temp_cidade1, temp_cidade2)
print(f"Estatística t: {estatistica:.2f}, p-valor: {p_valor:.3f}")

Saída: Estatística t: 1.83, p-valor: 0.104
```

Interpretação: O p-valor de 0,104 (maior que 0,05) não permite rejeitar H_0: com apenas cinco medições por cidade, a diferença observada entre as médias (22,2°C contra 20,0°C) pode ser fruto do acaso. Isso é diferente de provar que as cidades são iguais; uma amostra maior poderia revelar uma diferença real. Em um projeto de ML, esse tipo de teste ajuda a avaliar se uma *feature* como "cidade"carrega informação útil para prever a temperatura.

2.5 Normalização e Escalonamento

Ajustar a escala dos dados é uma etapa importante para boa parte dos modelos de *Machine Learning*, mas não para todos.

2.5.1 Por que Normalizar Dados para Machine Learning

A normalização (ex.: reescalar para o intervalo de 0 a 1) e a padronização (ex.: média 0 e desvio padrão 1) equalizam as magnitudes das *features*, evitando que variáveis com escalas maiores dominem o treinamento. Modelos baseados em distância (KNN, K-Means, SVM) e modelos treinados por gradiente descendente (regressão logística, redes neurais) são sensíveis à escala e dependem desse ajuste. Já os modelos baseados em regras de divisão, como árvores de decisão e Random Forest, são indiferentes à escala das variáveis.

2.5.2 Exemplo Prático: Escalonando Características para um Modelo

Escalonamos tamanho e quartos de casas:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

data = {'tamanho': [50, 70, 80, 100, 120], 'quartos': [1, 2, 2, 3, 3]}
df = pd.DataFrame(data)
scaler = StandardScaler()
dados_escalonados = scaler.fit_transform(df)
print("Dados Originais:")
print(df)
print("Dados Escalonados (média ~0, desvio ~1):")
print(pd.DataFrame(dados_escalonados, columns=['tamanho', 'quartos']))

Saída:
    Dados Originais:
       tamanho  quartos
    0       50        1
    1       70        2
    2       80        2
    3      100        3
    4      120        3
    Dados Escalonados (média ~0, desvio ~1):
        tamanho   quartos
    0 -1.406930 -1.603567
    1 -0.579324 -0.267261
    2 -0.165521 -0.267261
    3  0.662085  1.069045
    4  1.489691  1.069045
```

Interpretação: Antes do escalonamento, "tamanho"(50-120) tinha uma escala muito maior que "quartos"(1-3), o que poderia enviesar um modelo como SVM. Após o ajuste, ambas ficam na mesma ordem de grandeza e contribuem de forma equilibrada para o cálculo de distâncias.

2.6 Outliers em Modelos de Regressão

Em tarefas de regressão, a presença de *outliers*, valores extremos que desviam significativamente do padrão geral dos dados, pode comprometer a precisão e a confiabilidade dos modelos preditivos. Esses pontos anômalos surgem por diversos motivos, como erros

de medição, eventos raros ou variações naturais em datasets complexos. Em modelos de regressão linear, por exemplo, *outliers* exercem uma influência desproporcional na estimativa dos coeficientes, pois o método dos mínimos quadrados busca minimizar o erro total, dando peso excessivo a valores aberrantes. Já em árvores de decisão, embora menos sensíveis, *outliers* ainda podem distorcer as divisões iniciais, afetando a generalização.

Detectar e tratar *outliers* é uma etapa crítica no pré-processamento de dados para *Machine Learning*. Métodos comuns incluem a análise estatística baseada no desvio padrão (valores além de 2 ou 3 desvios da média são suspeitos) e o uso do intervalo interquartil (IQR), que identifica pontos fora dos limites definidos por $Q1 - 1{,}5 \times IQR$ e $Q3 + 1{,}5 \times IQR$, onde $Q1$ e $Q3$ são os quartis inferior e superior, e $IQR = Q3 - Q1$. Após a detecção, as opções de tratamento variam: remover os *outliers* (se forem erros), transformá-los (ex.: aplicar logaritmo) ou usar modelos robustos que os tolerem (ex.: regressão robusta ou árvores com poda).

A importância desse processo fica clara ao considerar seu impacto prático. Um único *outlier* em um dataset de preços de casas, como um imóvel de 60 m^2 custando R$1.000.000 devido a um erro de digitação, pode inclinar a reta de regressão linear, levando a previsões imprecisas para casas típicas. Identificar e mitigar esses casos garante que o modelo reflita as tendências reais dos dados, em vez de ser distorcido por anomalias.

2.6.1 Exemplo Prático: Detecção de Outliers em Preços de Casas

Criaremos um dataset fictício de preços de casas, incluindo um *outlier* intencional, para demonstrar como detectá-lo e avaliar seu impacto em uma regressão linear. O código Python a seguir implementa essa análise:

Usaremos, para o exemplo, `LinearRegression` do scikit-learn para ajustar a regressão e `matplotlib.pyplot` para visualização gráfica.

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
```

Criando o outlier e o dataset para exemplificação:

```
# O último par (tamanho=60, preco=1000000) é o outlier intencional
data = {
    'tamanho': [ 50, 70, 80, 100, 120, 60, 90, 110, 85, 95, 60 ],
    'preco': [ 180000, 240000, 280000, 350000, 420000, 200000, 300000,
    ↪  380000, 290000, 320000, 1000000 ]
}
df = pd.DataFrame(data)
```

Calculando os limites do IQR para identificar os outliers:

```
Q1 = df['preco'].quantile(0.25)
Q3 = df['preco'].quantile(0.75)
IQR = Q3 - Q1
limite_inferior = Q1 - 1.5 * IQR
limite_superior = Q3 + 1.5 * IQR

# Identificando outliers
outliers = df[(df['preco'] < limite_inferior) | (df['preco'] >
↪  limite_superior)]
print("Outliers detectados:")
print(outliers)
```

Saída:

- Outliers detectados:

tamanho	preco
60	1000000

Ajustando a regressão com e sem o outlier:

```
# Regressão linear com outliers
X = df[['tamanho']]
y = df['preco']
modelo_com_outlier = LinearRegression()
modelo_com_outlier.fit(X, y)
y_pred_com_outlier = modelo_com_outlier.predict(X)

# Removendo o outlier e treinando novamente
df_sem_outlier = df[df['preco'] <= limite_superior]
X_sem_outlier = df_sem_outlier[['tamanho']]
y_sem_outlier = df_sem_outlier['preco']
modelo_sem_outlier = LinearRegression()
modelo_sem_outlier.fit(X_sem_outlier, y_sem_outlier)
y_pred_sem_outlier = modelo_sem_outlier.predict(X_sem_outlier)
```

Visualização:

```
plt.scatter(X, y, color='blue', label='Dados com outlier')
plt.plot(X, y_pred_com_outlier,
        color='red',
        label='Regressão com outlier')
plt.scatter(X_sem_outlier,
        y_sem_outlier,
```

```
        color='green',
        label='Dados sem outlier')
plt.plot(X_sem_outlier,
        y_pred_sem_outlier,
        color='orange',
        label='Regressão sem outlier')
plt.xlabel('Tamanho (m²)')
plt.ylabel('Preço (R$)')
plt.title('Impacto de Outliers na Regressão Linear')
plt.legend()
plt.show()

# Coeficientes
print(f"Com outlier - Inclinação: {modelo_com_outlier.coef_[0]:.2f}, "
      f"Intercepto: {modelo_com_outlier.intercept_:.2f}")
print(f"Sem outlier - Inclinação: {modelo_sem_outlier.coef_[0]:.2f}, "
      f"Intercepto: {modelo_sem_outlier.intercept_:.2f}")
```

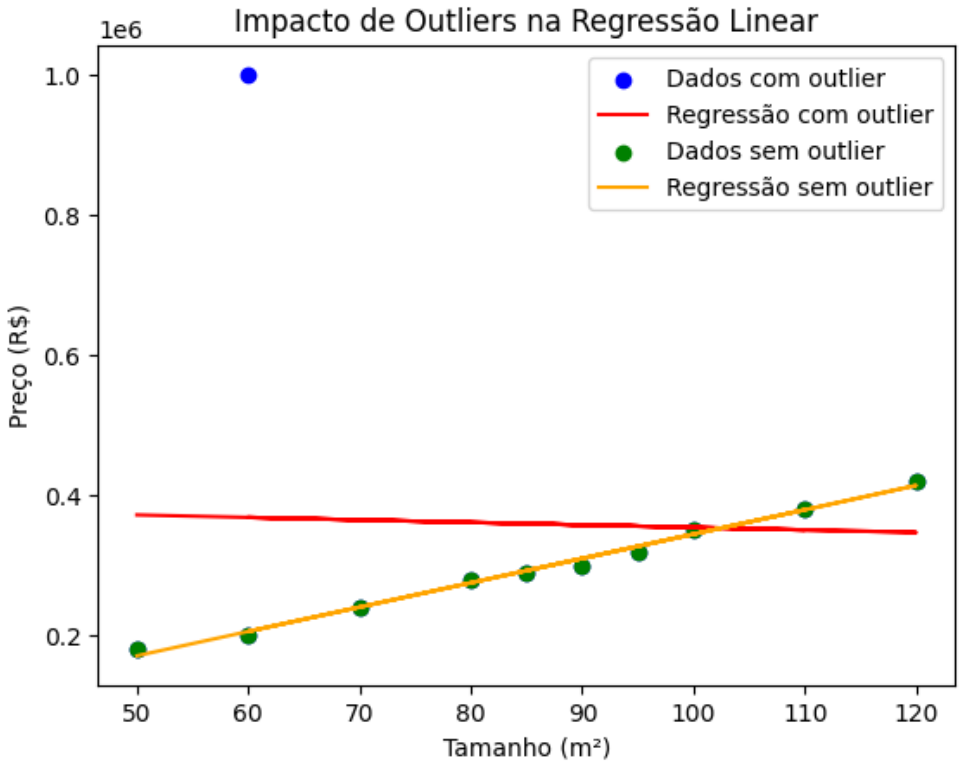


Figura 2.4: Representação de impacto de outliers na regressão linear.

- Coeficientes:

```
- Com outlier: Inclinação = -356.81, Intercepto = 389842.45
- Sem outlier: Inclinação = 3470.86, Intercepto = -2494.17
```

Gráfico: Na Figura 2.4, a reta ajustada com o *outlier* (vermelha) fica quase horizontal e desalinhada dos pontos típicos, enquanto a reta sem o *outlier* (laranja) segue de perto os dados verdes (sem o valor extremo).

2.6.2 Interpretação dos Resultados

A detecção via IQR identificou corretamente o preço de R$1.000.000 como um *outlier*, já que o limite superior calculado (R$522.500, com $Q1$ = R$260.000 e $Q3$ = R$365.000) foi amplamente ultrapassado. Esse valor, associado a um tamanho modesto de 60 m^2, é inconsistente com o padrão dos outros preços, que variam de R$180.000 a R$420.000.

A regressão linear com o *outlier* resultou em uma inclinação de $-356{,}81$: segundo esse modelo, cada metro quadrado adicional reduziria o preço em cerca de R$357, uma conclusão absurda. Como o ponto extremo pertence a uma casa pequena (60 m^2, abaixo do tamanho médio do dataset), ele ergue o lado esquerdo da reta e a deixa praticamente horizontal, com intercepto de 389842,45. Após removê-lo, a inclinação passa a 3470,86 (cerca de R$3.471 por m^2) e o intercepto fica próximo de zero ($-2494{,}17$), valores coerentes com os dados típicos. O gráfico confirma visualmente: a reta vermelha (com *outlier*) é puxada para cima pelo valor anômalo, enquanto a laranja (sem *outlier*) acompanha a tendência real.

Em *Machine Learning*, esse impacto é crítico. Um modelo treinado com *outliers* não tratados pode superestimar preços de casas pequenas e subestimar os de casas grandes, levando a previsões pouco confiáveis. Neste exemplo, o modelo com *outlier* prevê cerca de R$372 mil para a casa de 50 m^2 (preço real: R$180 mil) e R$347 mil para a de 120 m^2 (preço real: R$420 mil). Isso acontece porque os mínimos quadrados penalizam o erro ao quadrado, e um único desvio enorme passa a dominar a soma dos erros. A remoção do *outlier* melhorou a generalização, mas em cenários reais, outras estratégias, como substituir o valor por uma média ou usar um modelo robusto como RANSAC, poderiam ser exploradas. Para árvores de decisão, o *outlier* teria menos influência nas regiões finais, mas ainda poderia afetar divisões iniciais, reforçando a necessidade de tratamento prévio.

Esse exemplo destaca a importância de diagnosticar e lidar com *outliers* antes do treinamento, garantindo que o modelo capture as relações verdadeiras nos dados e produza previsões úteis em aplicações práticas, como avaliações imobiliárias.

Capítulo 3

Regressão: Previsão de Valores Contínuos

3.1 O que é Regressão?

Regressão é uma técnica de Machine Learning usada para prever valores contínuos, ou seja, números que podem assumir qualquer valor dentro de um intervalo. Exemplos incluem o preço de uma casa, a temperatura futura ou o tempo de entrega de um produto. Diferente da classificação, que prevê categorias discretas (como "sim"ou "não"), a regressão é ideal para problemas onde a saída é uma quantidade mensurável.

Em Machine Learning, a regressão é um tipo de aprendizado supervisionado, pois depende de um conjunto de dados rotulados contendo pares de entradas (variáveis independentes) e saídas (variáveis dependentes). O objetivo é encontrar uma função matemática que relacione essas entradas às saídas, permitindo previsões precisas para novos dados. Entre os métodos de regressão, o mais simples e amplamente utilizado é a regressão linear, que será o foco deste capítulo.

3.2 Regressão Linear

A regressão linear assume que a relação entre a variável de entrada (x) e a variável de saída (y) pode ser aproximada por uma reta. Na escola, essa reta aparece como $y = mx+b$. Em Machine Learning, o coeficiente angular costuma ser chamado de **peso** (*weight*) e escrito w, e o intercepto b recebe também o nome de **viés** (*bias*), sem relação com o viés estatístico de dados enviesados. Usaremos essa notação no resto do livro, porque ela se estende naturalmente para modelos com muitas variáveis e para as redes neurais da Parte II. A equação da regressão linear simples fica:

$$\hat{y} = wx + b \tag{3.1}$$

onde:

- $\hat{y}$: valor previsto (o "chapéu" distingue a previsão do valor real y);
- x: valor de entrada (variável independente);
- w: peso, ou coeficiente angular: quanto $\hat{y}$ aumenta quando x aumenta uma unidade;
- b: viés, ou intercepto: valor de $\hat{y}$ quando $x = 0$.

O objetivo do treinamento é ajustar w e b para minimizar o erro entre os valores previstos ($\hat{y}$) e os valores reais dos dados. Esse erro é frequentemente medido pelo erro quadrático médio (MSE), definido como:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{3.2}$$

onde y_i é o valor real, $\hat{y}_i$ é o valor previsto e n é o número de observações. A Figura 3.1 mostra uma reta ajustada a um conjunto de pontos.

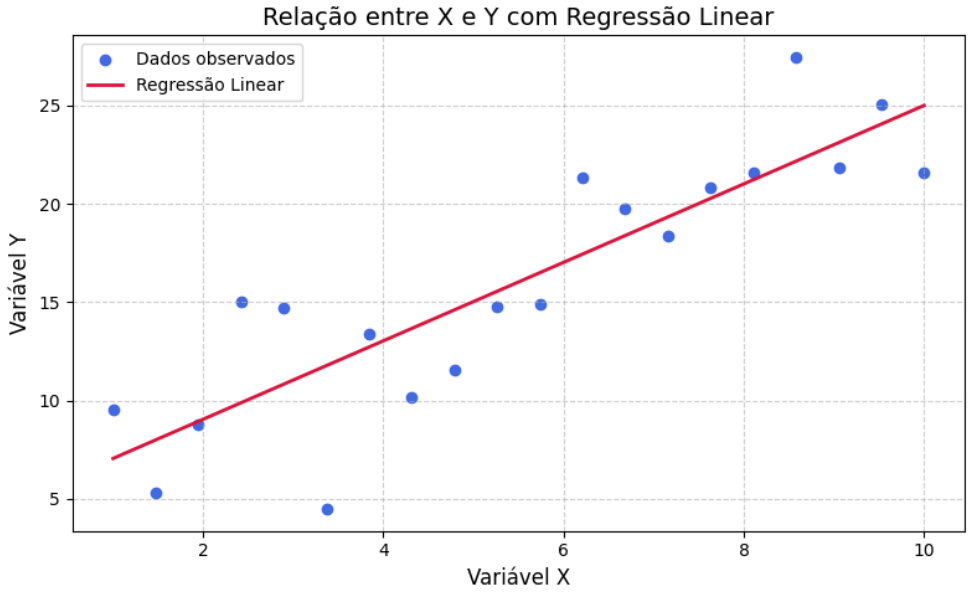


Figura 3.1: Representação gráfica de uma regressão linear.

3.3 Exemplo Prático: Prever o Preço de Casas

Para ilustrar a regressão linear, consideremos um exemplo prático: prever o preço de casas (em milhares de reais) com base em seu tamanho (em metros quadrados). Suponha os seguintes dados coletados:

Tamanho (m^2)	Preço (mil R$)
50	200
70	250
90	320
110	380
130	450

Tabela 3.1: Dados de tamanho e preço de casas.

3.3.1 Cálculo Manual Simplificado

Para uma regressão linear simples, podemos estimar w e b usando o método dos mínimos quadrados, que escolhe os valores com o menor MSE possível. A fórmula para o peso é:

$$w = \frac{n\sum(x_i y_i) - \sum x_i \sum y_i}{n\sum x_i^2 - (\sum x_i)^2} \tag{3.3}$$

e o viés:

$$b = \frac{\sum y_i - w\sum x_i}{n}. \tag{3.4}$$

Em ambas, n é o número de casas e cada somatório percorre todas elas ($i = 1, \ldots, n$).

Usando os dados da Tabela 3.1:

- $n = 5$,
- $\sum x_i = 50 + 70 + 90 + 110 + 130 = 450$,
- $\sum y_i = 200 + 250 + 320 + 380 + 450 = 1600$,
- $\sum x_i y_i = (50 \cdot 200) + (70 \cdot 250) + (90 \cdot 320) + (110 \cdot 380) + (130 \cdot 450) = 156.600$,
- $\sum x_i^2 = 50^2 + 70^2 + 90^2 + 110^2 + 130^2 = 44.500$.

Substituímos:

- $w = \frac{5 \cdot 156.600 - 450 \cdot 1600}{5 \cdot 44.500 - 450^2} = \frac{783.000 - 720.000}{222.500 - 202.500} = \frac{63.000}{20.000} = 3{,}15$,
- $b = \frac{1600 - 3{,}15 \cdot 450}{5} = \frac{1600 - 1417{,}5}{5} = \frac{182{,}5}{5} = 36{,}5$.

Assim, a equação é $\hat{y} = 3{,}15x + 36{,}5$.

3.3.2 Calculando com Python

Em Python, usamos a biblioteca `scikit-learn` para ajustar o modelo:

```
import numpy as np
from sklearn.linear_model import LinearRegression

# Dados Fictícios
X = np.array([[50], [70], [90], [110], [130]])  # Tamanho (m²)
y = np.array([200, 250, 320, 380, 450])         # Preço (mil R£)

# Cria e treina o modelo
modelo = LinearRegression()
modelo.fit(X, y)

# Coeficientes
w = modelo.coef_[0]       # Peso (inclinação)
b = modelo.intercept_     # Viés (intercepto)
print(f"Equação: y = {w:.2f}x + {b:.2f}")

# Previsão para uma casa de 100 m²
tamanho_teste = np.array([[100]])
preco_previsto = modelo.predict(tamanho_teste)
print(f"Preço previsto para 100 m²: {preco_previsto[0]:.2f} mil R$")
```

Saída:

```
Equação: y = 3.15x + 36.50
Preço previsto para 100 m²: 351.50 mil R$
```

3.3.3 Interpretação

A equação $\hat{y} = 3{,}15x + 36{,}5$, idêntica à do cálculo manual, indica que cada metro quadrado adicional aumenta o preço em 3,15 mil reais, com uma base de 36,5 mil reais. Para uma casa de 100 m², o preço previsto é 351,50 mil reais, um valor coerente com os dados (entre os 320 mil da casa de 90 m² e os 380 mil da de 110 m²). A regressão linear é útil para previsões simples, mas seu desempenho depende da linearidade da relação entre as variáveis.

3.4 Notação Algébrica e Vetorial

O preço de uma casa raramente depende só do tamanho. Número de quartos, idade do imóvel e bairro também pesam, e cada nova variável acrescentaria mais termos às fórmulas

da seção anterior. Para trabalhar com qualquer número de variáveis sem reescrever tudo, a regressão linear (e praticamente todo o Machine Learning) usa a linguagem de vetores e matrizes. Esta seção constrói essa linguagem passo a passo, sempre com o exemplo das casas. Ela será reaproveitada na classificação (Capítulo 4), no clustering e nas redes neurais da Parte II.

3.4.1 Convenções

A Tabela 3.2 reúne os símbolos usados daqui em diante. A regra principal é visual: letra normal para um número, minúscula em negrito para vetor e maiúscula em negrito para matriz.

Símbolo	Significado
x, w, b	Minúsculas em itálico: números isolados (escalares).
$\mathbf{x}$, $\mathbf{w}$	Minúsculas em negrito: vetores, escritos como colunas.
$\mathbf{X}$	Maiúscula em negrito: matriz.
n	Número de exemplos (linhas do dataset), indexados por $i = 1, \ldots, n$.
d	Número de *features* (colunas), indexadas por $j = 1, \ldots, d$.
x_{ij}	Valor da *feature* j no exemplo i.
$\mathbf{x}_i$	Vetor com todas as *features* do exemplo i.
y_i, $\hat{y}_i$	Valor real e valor previsto para o exemplo i.
$\mathbf{v}^\top$	Transposta de $\mathbf{v}$: transforma uma coluna em linha, e vice-versa.
$\mathbb{R}^d$	Conjunto dos vetores formados por d números reais.
$\|\mathbf{v}\|$	Norma (comprimento) do vetor $\mathbf{v}$.

Tabela 3.2: Notação usada a partir deste capítulo.

3.4.2 Regressão Linear Múltipla

Com d *features*, o modelo ganha um peso para cada uma:

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b = \sum_{j=1}^{d} w_j x_j + b \tag{3.5}$$

Cada *feature* x_j é multiplicada pelo seu peso w_j, que mede quanto a previsão muda quando x_j aumenta uma unidade e todas as outras *features* ficam constantes. O viés b continua sendo um só. Com $d = 1$, a equação volta a ser a reta $\hat{y} = wx + b$.

Para as casas, com o tamanho como x_1 e o número de quartos como x_2:

$$\hat{y} = w_1 \cdot \text{tamanho} + w_2 \cdot \text{quartos} + b$$

Com duas *features*, o modelo deixa de ser uma reta e passa a ser um plano no espaço (tamanho, quartos, preço). Com mais *features*, vira um **hiperplano**: não conseguimos desenhá-lo, mas ele segue exatamente as mesmas regras.

3.4.3 Vetores e Produto Escalar

Agrupamos as *features* de um exemplo em um vetor $\mathbf{x} \in \mathbb{R}^d$ e os pesos em um vetor $\mathbf{w} \in \mathbb{R}^d$:

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{pmatrix}, \qquad \mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{pmatrix}$$

O **produto escalar** entre dois vetores do mesmo tamanho multiplica-os elemento a elemento e soma os resultados:

$$\mathbf{w}^\top \mathbf{x} = \begin{pmatrix} w_1 & w_2 & \cdots & w_d \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{pmatrix} = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d \tag{3.6}$$

A transposta $\mathbf{w}^\top$ deita o vetor de pesos, e uma linha multiplicada por uma coluna resulta em um único número. Com isso, a Equação 3.5 fica:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b \tag{3.7}$$

Essa linha resume a regressão linear com qualquer número de *features*: o somatório continua lá, escondido dentro do produto escalar.

Ajustando o modelo de tamanho e quartos às 10 casas que usaremos também na árvore de decisão (Seção 3.5), com preços em mil reais, obtemos $\mathbf{w} = (3{,}0816;\ 12{,}4633)$ e $b = 4{,}8124$ (o código está mais adiante, na Seção 3.4.7). Para prever o preço de uma casa de 100 m² com 3 quartos:

$$\hat{y} = \begin{pmatrix} 3{,}0816 & 12{,}4633 \end{pmatrix} \begin{pmatrix} 100 \\ 3 \end{pmatrix} + 4{,}8124 = 308{,}16 + 37{,}39 + 4{,}81 = 350{,}36$$

O produto escalar é uma soma ponderada: cada *feature* entra com seu valor multiplicado pelo peso que o modelo lhe atribui. Nessa previsão, o tamanho contribui com 308,16 mil reais e os quartos com 37,39 mil.

3.4.4 A Matriz de Dados

Empilhando os n exemplos, um por linha, obtemos a **matriz de dados** $\mathbf{X}$, com n linhas e d colunas. É o mesmo formato de um `DataFrame` do pandas: uma linha por casa,

uma coluna por *feature*.

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^\top \\ \mathbf{x}_2^\top \\ \vdots \\ \mathbf{x}_n^\top \end{pmatrix} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nd} \end{pmatrix}$$

O produto de uma matriz por um vetor calcula o produto escalar de cada linha com o vetor. Assim, as previsões para todos os exemplos saem de uma vez:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b\,\mathbf{1} \tag{3.8}$$

onde $\hat{\mathbf{y}}$ é o vetor com as n previsões, a i-ésima linha de $\mathbf{X}\mathbf{w}$ é $\mathbf{x}_i^\top \mathbf{w}$ e $\mathbf{1}$ é um vetor de n uns, que soma o mesmo b a todas as previsões.

Um truque simplifica ainda mais a escrita: acrescentar a $\mathbf{X}$ uma primeira coluna formada só por uns e colocar o viés no início do vetor de parâmetros, $\boldsymbol{\theta} = (b;\ w_1;\ \ldots;\ w_d)$. A coluna de uns multiplica b em todas as linhas, e o modelo inteiro vira um único produto:

$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta} \tag{3.9}$$

Daqui até o fim do capítulo, $\mathbf{X}$ inclui essa coluna de uns. Para as 5 casas da Tabela 3.1 ($d = 1$), com os parâmetros encontrados no cálculo manual:

$$\mathbf{X} = \begin{pmatrix} 1 & 50 \\ 1 & 70 \\ 1 & 90 \\ 1 & 110 \\ 1 & 130 \end{pmatrix}, \quad \boldsymbol{\theta} = \begin{pmatrix} 36{,}5 \\ 3{,}15 \end{pmatrix}, \quad \hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta} = \begin{pmatrix} 36{,}5 + 3{,}15 \cdot 50 \\ 36{,}5 + 3{,}15 \cdot 70 \\ 36{,}5 + 3{,}15 \cdot 90 \\ 36{,}5 + 3{,}15 \cdot 110 \\ 36{,}5 + 3{,}15 \cdot 130 \end{pmatrix} = \begin{pmatrix} 194 \\ 257 \\ 320 \\ 383 \\ 446 \end{pmatrix}$$

3.4.5 A Função de Custo em Forma Vetorial

O vetor de **resíduos** guarda o erro cometido em cada exemplo:

$$\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \mathbf{X}\boldsymbol{\theta}$$

Para as casas:

$$\mathbf{e} = \begin{pmatrix} 200 \\ 250 \\ 320 \\ 380 \\ 450 \end{pmatrix} - \begin{pmatrix} 194 \\ 257 \\ 320 \\ 383 \\ 446 \end{pmatrix} = \begin{pmatrix} 6 \\ -7 \\ 0 \\ -3 \\ 4 \end{pmatrix}$$

O MSE é a soma dos quadrados dos resíduos dividida por n. A soma dos quadrados dos componentes de um vetor é o produto escalar dele com ele mesmo, $\mathbf{e}^\top \mathbf{e}$, que também é o

quadrado do seu comprimento, a **norma euclidiana** $\|\mathbf{e}\|$. Assim, o MSE vira uma função dos parâmetros:

$$J(\boldsymbol{\theta}) = \frac{1}{n}\|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2 = \frac{1}{n}(\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) \tag{3.10}$$

J é a **função de custo**: para cada escolha de $\boldsymbol{\theta}$, ela devolve um número que mede o quanto o modelo erra nos dados de treino. Nas casas, $\mathbf{e}^\top\mathbf{e} = 36 + 49 + 0 + 9 + 16 = 110$ e $J = 110/5 = 22$.

Treinar o modelo é encontrar o $\boldsymbol{\theta}$ que torna J o menor possível. Essa formulação (escolher um modelo, escolher uma função de custo e minimizá-la) é a mesma das redes neurais da Parte II; só mudam o modelo e o custo.

3.4.6 A Equação Normal

Para a regressão linear, o mínimo de J tem solução exata. Igualando a zero as derivadas de J em relação a cada parâmetro (ferramenta que a Parte II desenvolve com calma), chega-se ao sistema $\mathbf{X}^\top\mathbf{X}\,\boldsymbol{\theta} = \mathbf{X}^\top\mathbf{y}$, cuja solução é a **equação normal**:

$$\boldsymbol{\theta} = (\mathbf{X}^\top\mathbf{X})^{-1}\,\mathbf{X}^\top\mathbf{y} \tag{3.11}$$

Os termos são:

- $\mathbf{X}^\top\mathbf{X}$: matriz quadrada, com uma linha e uma coluna por parâmetro, que resume como as colunas de $\mathbf{X}$ se relacionam entre si;
- $\mathbf{X}^\top\mathbf{y}$: vetor que resume como cada coluna de $\mathbf{X}$ se relaciona com o alvo;
- $(\cdot)^{-1}$: a inversa da matriz, o equivalente matricial de dividir.

Vamos resolvê-la à mão para as 5 casas. Primeiro, os dois produtos:

$$\mathbf{X}^\top\mathbf{X} = \begin{pmatrix} n & \sum x_i \\ \sum x_i & \sum x_i^2 \end{pmatrix} = \begin{pmatrix} 5 & 450 \\ 450 & 44.500 \end{pmatrix}, \qquad \mathbf{X}^\top\mathbf{y} = \begin{pmatrix} \sum y_i \\ \sum x_i y_i \end{pmatrix} = \begin{pmatrix} 1600 \\ 156.600 \end{pmatrix}$$

São exatamente os somatórios do cálculo manual, agora organizados em uma matriz e um vetor. Para inverter uma matriz 2×2, usamos a regra

$$\begin{pmatrix} p & q \\ r & s \end{pmatrix}^{-1} = \frac{1}{ps - qr}\begin{pmatrix} s & -q \\ -r & p \end{pmatrix}$$

em que $ps - qr$ é o determinante da matriz. Aqui ele vale $5 \cdot 44.500 - 450^2 = 20.000$, o mesmo denominador da Equação 3.3. Então:

$$\begin{aligned} \boldsymbol{\theta} &= \frac{1}{20.000}\begin{pmatrix} 44.500 & -450 \\ -450 & 5 \end{pmatrix}\begin{pmatrix} 1600 \\ 156.600 \end{pmatrix} \\ &= \frac{1}{20.000}\begin{pmatrix} 44.500 \cdot 1600 - 450 \cdot 156.600 \\ -450 \cdot 1600 + 5 \cdot 156.600 \end{pmatrix} \\ &= \frac{1}{20.000}\begin{pmatrix} 730.000 \\ 63.000 \end{pmatrix} = \begin{pmatrix} 36{,}5 \\ 3{,}15 \end{pmatrix} \end{aligned}$$

Chegamos a $b = 36{,}5$ e $w = 3{,}15$, como antes. As fórmulas do cálculo manual são a equação normal escrita por extenso para o caso de uma única *feature*; a versão matricial vale para qualquer d.

A solução tem uma propriedade geométrica útil: $\mathbf{X}^\top \mathbf{e} = \mathbf{0}$, ou seja, o vetor de resíduos tem produto escalar zero com cada coluna de $\mathbf{X}$ (dizemos que é **ortogonal** a elas). Com a coluna de uns, isso significa que os resíduos somam zero: $6 - 7 + 0 - 3 + 4 = 0$. Com a coluna de tamanhos, $50 \cdot 6 - 70 \cdot 7 + 90 \cdot 0 - 110 \cdot 3 + 130 \cdot 4 = 300 - 490 + 0 - 330 + 520 = 0$. A intuição: se os resíduos ainda tivessem alguma relação linear com uma das colunas, ajustar o parâmetro dessa coluna reduziria o erro, e a solução não seria a melhor.

3.4.7 Equação Normal em Python

Em NumPy, o operador `@` faz produtos de matrizes e `.T` calcula a transposta. O código a seguir refaz o exemplo das 5 casas:

```
import numpy as np

# Casas da Tabela 3.1: tamanho (m²) e preço (mil R£)
x = np.array([50, 70, 90, 110, 130])
y = np.array([200, 250, 320, 380, 450])

# Matriz de dados: coluna de uns (viés) + coluna de tamanhos
X = np.column_stack([np.ones(len(x)), x])

# Equação normal: resolve (X^T X) theta = X^T y
theta = np.linalg.solve(X.T @ X, X.T @ y)
print("theta = [b, w]:", np.round(theta, 2))

# Previsões em lote e resíduos
y_hat = X @ theta
residuos = y - y_hat
print("Previsões:", np.round(y_hat, 2))
print("Resíduos: ", np.round(residuos, 2))
print("MSE:", round(residuos @ residuos / len(y), 2))
```

```
Saída: theta = [b, w]: [36.5   3.15]
Previsões: [194. 257. 320. 383. 446.]
Resíduos:  [ 6. -7.  0. -3.  4.]
MSE: 22.0
```

Usamos `np.linalg.solve`, que resolve o sistema $\mathbf{X}^\top \mathbf{X}\, \boldsymbol{\theta} = \mathbf{X}^\top \mathbf{y}$ diretamente, em vez de calcular a inversa com `np.linalg.inv` e multiplicar. O resultado é o mesmo, com menos operações e menos erro de arredondamento.

O mesmo código funciona com duas *features*; basta acrescentar uma coluna à matriz. Abaixo, ajustamos o modelo de tamanho e quartos às 10 casas e comparamos o resultado com o do scikit-learn:

```
from sklearn.linear_model import LinearRegression

tamanho = np.array([50, 70, 80, 100, 120, 60, 90, 110, 85, 95])
quartos = np.array([1, 2, 2, 3, 3, 1, 2, 3, 2, 2])
preco = np.array([180, 240, 280, 350, 420, 200, 300, 380, 290, 320])  # mil R£

# Equação normal com duas features
X = np.column_stack([np.ones(len(preco)), tamanho, quartos])
theta = np.linalg.solve(X.T @ X, X.T @ preco)
b, w = theta[0], theta[1:]
print(f"NumPy:        b = {b:.2f}, w = {np.round(w, 2)}")

# Mesmo modelo com scikit-learn (sem a coluna de uns)
modelo = LinearRegression().fit(np.column_stack([tamanho, quartos]), preco)
print(f"scikit-learn: b = {modelo.intercept_:.2f}, w = {np.round(modelo.coef_, 2)}")

# Previsão: produto escalar w^T x + b
x_novo = np.array([100, 3])
print(f"Previsão para 100 m² e 3 quartos: {w @ x_novo + b:.2f} mil R$")
```

```
Saída: NumPy:        b = 4.81, w = [ 3.08 12.46]
scikit-learn: b = 4.81, w = [ 3.08 12.46]
Previsão para 100 m² e 3 quartos: 350.36 mil R$
```

Os dois caminhos chegam aos mesmos parâmetros. O `LinearRegression` acrescenta o viés por conta própria e usa um método de mínimos quadrados numericamente mais estável que a equação normal, mas resolve o mesmo problema. A última linha é a Equação 3.7 em código: `w @ x_novo` é o produto escalar $\mathbf{w}^\top \mathbf{x}$.

3.4.8 Interpretando os Pesos

O modelo ajustado às 10 casas é:

$$\hat{y} = 4{,}81 + 3{,}08 \cdot \text{tamanho} + 12{,}46 \cdot \text{quartos} \quad \text{(mil R\$)}$$

O peso $w_1 = 3{,}08$ diz que, com o número de quartos fixo, cada metro quadrado a mais soma cerca de 3,08 mil reais à previsão. O peso $w_2 = 12{,}46$ diz que, com o tamanho fixo, cada quarto a mais soma 12,46 mil reais.

"Fixo" é a palavra que importa. Ajustando às mesmas 10 casas um modelo só com o tamanho, o peso do tamanho é 3,47. Quando os quartos entram, parte do efeito que

antes era atribuído ao tamanho passa para eles, porque as duas *features* andam juntas: a correlação de Pearson (Capítulo 2) entre tamanho e quartos nessas casas é 0,92.

Essa correlação forte entre *features*, chamada **multicolinearidade**, deixa os pesos instáveis. Nas 10 casas quase não há imóveis de mesmo tamanho com número diferente de quartos, então o modelo tem pouca informação para separar um efeito do outro. Removendo uma casa de cada vez e reajustando o modelo, o peso dos quartos varia de 9,76 a 17,41, enquanto o do tamanho fica entre 2,88 e 3,22. A previsão do preço continua boa; o que fica pouco confiável é a leitura isolada de w_2.

Também vale notar que o MSE no próprio treino caiu de 35,89 (só tamanho) para 24,79 (tamanho e quartos). Isso não prova que o modelo melhorou: na regressão linear, acrescentar uma *feature* nunca aumenta o erro de treino, porque o modelo sempre pode dar peso zero a ela. Medir o desempenho em dados que o modelo não viu é o assunto do Capítulo 6.

3.4.9 Limitações da Solução Fechada

A equação normal é elegante, mas tem limites práticos:

- **Invertibilidade:** $\mathbf{X}^\top\mathbf{X}$ precisa ter inversa. Isso falha quando uma coluna é combinação exata de outras (por exemplo, a área em m^2 e a mesma área em pés quadrados) ou quando há mais parâmetros do que exemplos. Com colunas quase redundantes, como tamanho e quartos, a inversa existe, mas pequenas mudanças nos dados mudam muito o resultado.

- **Escala:** colunas em escalas muito diferentes (uns, tamanhos de 50 a 120, quartos de 1 a 3) pioram a precisão numérica e impedem comparar os pesos entre si. Padronizar as *features*, como no Capítulo 2, resolve as duas coisas.

- **Custo computacional:** resolver o sistema exige um número de operações que cresce aproximadamente com o cubo do número de parâmetros. Com milhares ou milhões de *features*, como em textos e imagens, isso se torna inviável. Nesses casos, os parâmetros são encontrados aos poucos pelo **gradiente descendente**, assunto do Capítulo 9.

- **Forma do modelo:** $\hat{y} = \mathbf{w}^\top\mathbf{x} + b$ só representa relações lineares nas *features* fornecidas. Para capturar curvas, é preciso criar novas *features* (por exemplo, x^2) ou usar outro tipo de modelo, como a árvore de decisão da próxima seção.

- **Outliers:** como o custo eleva os erros ao quadrado, um único ponto extremo pode puxar o hiperplano inteiro, como vimos no Capítulo 2.

3.5 Árvore de Decisão

3.5.1 Árvores de Decisão para Regressão

Embora a regressão linear seja uma abordagem clássica para prever valores contínuos, as árvores de decisão oferecem uma alternativa poderosa e flexível, especialmente em cenários com relações não lineares entre variáveis. Este método, parte do algoritmo CART (*Classification and Regression Tree*), constrói uma estrutura hierárquica que divide o espaço de dados em regiões com base em condições sobre os atributos, prevendo o valor médio das amostras em cada região terminal. Diferentemente da regressão linear, que assume uma relação linear global, as árvores de decisão capturam interações locais e padrões complexos, tornando-as adequadas para datasets com comportamento não uniforme.

O funcionamento de uma árvore de decisão para regressão começa com a seleção de uma *feature* e um ponto de corte que minimizem o erro quadrático médio (MSE, do inglês *Mean Squared Error*) entre os valores reais e a média da região resultante. Por exemplo, em um dataset com tamanho de casas e número de quartos como preditores de preço, a árvore pode perguntar: "O tamanho é maior que 75 m^2?"Se sim, segue para um ramo; se não, para outro. Esse processo de divisão continua recursivamente, criando nós e ramos, até que um critério de parada seja atingido, como uma profundidade máxima ou um número mínimo de amostras por nó. O valor previsto em cada folha é a média dos valores alvo das amostras ali contidas.

As árvores de decisão têm vantagens notáveis: são intuitivas, fáceis de interpretar e não requerem suposições sobre a distribuição dos dados. Contudo, elas podem superajustar (*overfitting*) se não forem limitadas, especialmente em datasets com ruído ou muitas variáveis. Parâmetros como `max_depth` (profundidade máxima) e `min_samples_split` (mínimo de amostras por divisão) ajudam a controlar esse risco, equilibrando flexibilidade e generalização.

Exemplo Prático: Previsão de Preços de Casas

Para ilustrar o uso de árvores de decisão em regressão, consideremos um dataset fictício de preços de casas, com duas *features*: tamanho (em metros quadrados) e número de quartos. Nosso objetivo é prever o preço com base nessas variáveis. O código Python a seguir implementa o modelo utilizando a biblioteca `scikit-learn`:

```
import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Criando dados fictícios
data = {
```

```
    'tamanho': [50, 70, 80, 100, 120, 60, 90, 110, 85, 95],
    'quartos': [1, 2, 2, 3, 3, 1, 2, 3, 2, 2],
    'preco': [180000, 240000, 280000, 350000,
              420000, 200000, 300000, 380000, 290000, 320000]
}
df = pd.DataFrame(data)

# Separando features (X) e alvo (y)
X = df[['tamanho', 'quartos']]
y = df['preco']

# Dividindo em conjuntos de treino (80%) e teste (20%)
X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size=0.2,
                                                    random_state=42)

# Inicializando e treinando o modelo de árvore de decisão
modelo = DecisionTreeRegressor(max_depth=3, random_state=42)
modelo.fit(X_train, y_train)

# Fazendo previsões no conjunto de teste
y_pred = modelo.predict(X_test)

# Calculando o erro quadrático médio
mse = mean_squared_error(y_test, y_pred)
print(f"Erro Quadrático Médio (MSE): {mse:.2f}")

# Exibindo as previsões e valores reais
resultados = pd.DataFrame({'Real': y_test, 'Previsto': y_pred})
print("Comparação de Valores Reais e Previstos:")
print(resultados)
```

Resultados: Com `random_state=42`, o conjunto de teste ficou com as amostras (tamanho=85, quartos=2) e (tamanho=70, quartos=2). A saída é:

```
Erro Quadrático Médio (MSE): 850000000.00
Comparação de Valores Reais e Previstos:
     Real  Previsto
8  290000  280000.0
1  240000  200000.0
```

Interpretação dos Resultados

O MSE de 850000000 é a média dos erros ao quadrado entre os preços reais e previstos no conjunto de teste. Convertendo para RMSE ($\sqrt{850000000} \approx 29155$), vemos que o erro típico é de cerca de R$29.000, razoável para preços que variam de R$180.000 a R$420.000, mas longe de desprezível. Na comparação, a casa de 85 m^2 foi prevista com erro de R$10.000, enquanto a de 70 m^2 recebeu R$200.000, R$40.000 abaixo do valor real: sem ela no treino, a árvore a colocou na mesma folha da casa de 60 m^2.

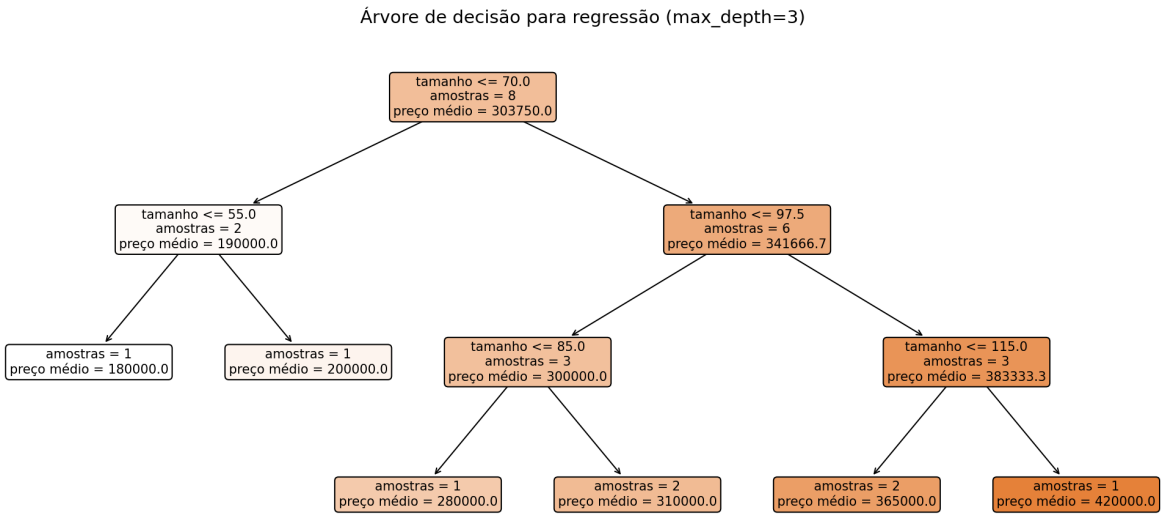


Figura 3.2: Árvore de decisão treinada no exemplo (gerada com `sklearn.tree.plot_tree`). Cada nó mostra a regra de divisão, o número de amostras e o preço médio; as folhas trazem o valor previsto.

A Figura 3.2 mostra as divisões aprendidas: a primeira pergunta é "tamanho $\leq$ 70 m^2?", separando as casas menores (preços mais baixos) das maiores, e os níveis seguintes refinam os cortes de tamanho. A *feature* quartos não foi usada em nenhuma divisão, porque neste dataset ela acompanha o tamanho e não acrescenta informação. O parâmetro `max_depth=3` limitou a árvore a três níveis de decisão, evitando que ela memorize os dados de treino (sobreajuste) e promovendo generalização. Em *Machine Learning*, esse controle é crucial: uma árvore muito profunda poderia alcançar MSE zero no treino, mas falhar miseravelmente no teste.

Comparada à regressão linear, a árvore de decisão tem a vantagem de modelar relações não lineares, por exemplo, o impacto do número de quartos pode variar dependendo do tamanho da casa, algo que uma linha reta não captura. Contudo, seu desempenho depende do tamanho do dataset: com apenas 10 amostras, os resultados são preliminares. Em datasets maiores, como milhares de casas, a árvore poderia revelar interações mais complexas, mas também exigiria ajustes adicionais (ex.: poda ou aumento de `min_samples_split`).

Em um contexto prático, esse modelo poderia ser usado por uma imobiliária para estimar preços iniciais de imóveis à venda, fornecendo uma base interpretável para ajustes manuais. Para melhorar, poderíamos testar profundidades diferentes ou combinar com outros métodos (ex.: Random Forest), tópicos que ficam para estudos posteriores.

3.6 Avaliação de Modelos de Regressão

Para avaliar a performance de modelos de regressão, utilizamos métricas que quantificam o erro entre os valores previstos e os reais. As mais comuns são o Erro Quadrático Médio (*Mean Squared Error*, MSE), a Raiz do Erro Quadrático Médio (*Root Mean Squared Error*, RMSE), o Erro Absoluto Médio (*Mean Absolute Error*, MAE) e o coeficiente de determinação (R^2), descritas na Tabela 3.3.

Métrica	**Descrição**	**Fórmula**	**Interpretação**
MSE	Média dos quadrados dos erros entre valores previstos e reais	$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$	Valores menores indicam melhor ajuste; sensível a outliers devido ao quadrado.
RMSE	Raiz quadrada do MSE	$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$	Mesma unidade dos dados (ex.: R$); mais interpretável que o MSE.
MAE	Média dos erros em valor absoluto	$\frac{1}{n}\sum_{i=1}^{n}\lvert y_i - \hat{y}_i\rvert$	Mesma unidade dos dados; menos sensível a outliers que o MSE.
R^2	Proporção da variância de y explicada pelo modelo	$1 - \frac{\sum_{i=1}^{n}(y_i-\hat{y}_i)^2}{\sum_{i=1}^{n}(y_i-\bar{y})^2}$	Quanto mais próximo de 1, melhor; 0 equivale a prever sempre a média.

Tabela 3.3: Métricas de avaliação para modelos de regressão. Aqui, y_i é o valor real, $\hat{y}_i$ é o valor previsto, $\bar{y}$ é a média dos valores reais e n é o número de observações.

Capítulo 4

Classificação: Previsão de Categorias

4.1 O que é Classificação?

Classificação é uma tarefa fundamental em Machine Learning que consiste em prever a categoria ou classe à qual um dado pertence, com base em suas características. Diferente da regressão, que estima valores contínuos (como o preço de uma casa), a classificação lida com variáveis discretas, geralmente representadas como rótulos ou etiquetas. O objetivo é treinar um modelo com exemplos rotulados, dados onde a categoria correta é conhecida, para que ele possa classificar novos dados desconhecidos de forma precisa.

Em aplicações reais, a classificação está presente em diversas áreas. Um exemplo clássico é a detecção de spam em e-mails: dado o conteúdo de uma mensagem, o modelo deve decidir se ela é "spam"ou "não spam". Outros casos incluem o diagnóstico médico (doente/saudável), reconhecimento de imagens (gato/cachorro) e análise de sentimentos (positivo/negativo). Esses problemas requerem que o modelo aprenda padrões nos dados, como palavras específicas em e-mails ou sintomas em pacientes, para realizar previsões confiáveis.

4.2 Modelos Básicos de Classificação

Dois modelos servem de porta de entrada para a classificação: a Regressão Logística e o K-Nearest Neighbors (KNN). Ambos usam a notação vetorial da Seção 3.4 de formas diferentes: o primeiro calcula um produto escalar, o segundo mede distâncias.

4.2.1 Regressão Logística

Apesar do nome, a regressão logística é um modelo de classificação. Ela estima a probabilidade de um exemplo pertencer à classe positiva (classe 1) em problemas binários. O ponto de partida é a mesma combinação linear da regressão (Equação 3.7):

$$z = \mathbf{w}^\top \mathbf{x} + b = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b \tag{4.1}$$

O valor z, chamado de **logit** ou escore, pode ser qualquer número real, e por isso não serve como probabilidade. A **função sigmoide** σ comprime esse valor no intervalo entre 0 e 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{4.2}$$

em que $e \approx 2{,}718$ é a base do logaritmo natural. Juntando as duas partes, o modelo completo é:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \tag{4.3}$$

onde $P(y = 1 \mid \mathbf{x})$ se lê "probabilidade de y ser 1, dadas as *features* $\mathbf{x}$". A probabilidade da classe 0 é o complemento, $1 - P(y = 1 \mid \mathbf{x})$.

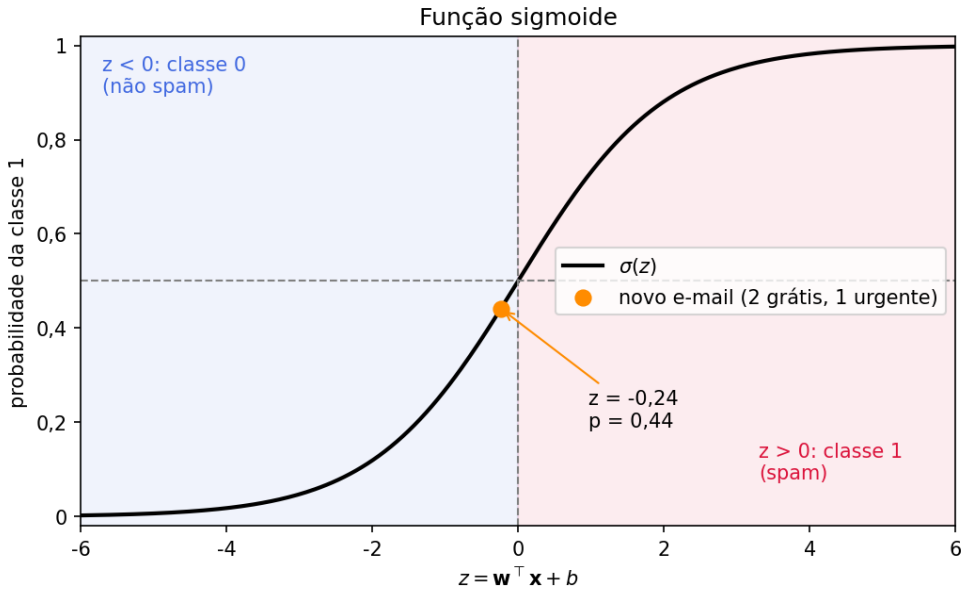


Figura 4.1: A função sigmoide transforma o escore z em probabilidade. O ponto laranja é o novo e-mail do exemplo desta seção, com $z = -0{,}24$ e probabilidade de spam de 0,44.

A Figura 4.1 mostra o formato em S da sigmoide. Três propriedades explicam seu uso:

- $\sigma(0) = 0{,}5$: escore zero significa indecisão total;
- para z muito positivo, e^{-z} fica próximo de zero e $\sigma(z)$ se aproxima de 1; para z muito negativo, e^{-z} fica enorme e $\sigma(z)$ se aproxima de 0;
- a curva é simétrica: $\sigma(-z) = 1 - \sigma(z)$.

O exemplo é classificado como classe 1 quando $P(y = 1 \mid \mathbf{x}) > 0{,}5$. Como $\sigma(z) > 0{,}5$ exatamente quando $z > 0$, a regra de decisão se resume ao sinal do escore:

$$\hat{y} = \begin{cases} 1, & \text{se } \mathbf{w}^\top \mathbf{x} + b > 0 \\ 0, & \text{caso contrário} \end{cases} \tag{4.4}$$

A sigmoide traduz o escore em probabilidade, mas a classe prevista depende apenas de z ser positivo ou negativo.

Por exemplo, considere a tarefa de classificar e-mails como spam ou não spam com base em duas *features*: o número de ocorrências das palavras "grátis" (x_1) e "urgente" (x_2). Um conjunto de dados fictício poderia ser:

- E-mail 1: 2 "grátis", 0 "urgente" → Não spam,
- E-mail 2: 1 "grátis", 3 "urgente" → Spam,
- E-mail 3: 0 "grátis", 1 "urgente" → Não spam,
- E-mail 4: 3 "grátis", 2 "urgente" → Spam.

A regressão logística ajusta $\mathbf{w}$ e b para prever a probabilidade de spam. Abaixo, o exemplo em Python com o scikit-learn:

```
from sklearn.linear_model import LogisticRegression
import numpy as np

# Dados: [nº de "grátis", nº de "urgente"]
X = np.array([[2, 0], [1, 3], [0, 1], [3, 2]])
y = [0, 1, 0, 1]  # 0 = Não spam, 1 = Spam

# Treina o modelo
modelo = LogisticRegression()
modelo.fit(X, y)

# Parâmetros aprendidos
w = modelo.coef_[0]
b = modelo.intercept_[0]
print("w =", np.round(w, 3), " b =", round(b, 3))

# Previsão para novo e-mail: 2 "grátis", 1 "urgente"
novo_email = np.array([[2, 1]])
probabilidade = modelo.predict_proba(novo_email)
print(f"Probabilidade de spam: {probabilidade[0][1]:.2f}")
```

```
Saída:
    w = [0.471 0.942]  b = -2.12
    Probabilidade de spam: 0.44
```

A probabilidade de 0,44 fica abaixo de 0,5, então o e-mail é classificado como não spam.

Conferindo a Conta à Mão

Com $\mathbf{w} = (0{,}471;\ 0{,}942)$, $b = -2{,}1195$ (a saída arredonda para $-2{,}12$) e o novo e-mail $\mathbf{x} = (2;\ 1)$, o escore é:

$$z = \mathbf{w}^\top \mathbf{x} + b = 0{,}471 \cdot 2 + 0{,}942 \cdot 1 - 2{,}1195 = 0{,}942 + 0{,}942 - 2{,}1195 = -0{,}2355$$

e a probabilidade:

$$P(y = 1 \mid \mathbf{x}) = \sigma(-0{,}2355) = \frac{1}{1 + e^{0{,}2355}} = \frac{1}{1 + 1{,}2655} \approx 0{,}44$$

O mesmo valor do `predict_proba`. Como z é negativo, a classe prevista é 0 (não spam).

Para calcular as probabilidades de vários exemplos de uma vez, usamos a forma matricial: $\mathbf{p} = \sigma(\mathbf{X}\mathbf{w} + b\,\mathbf{1})$, com a sigmoide aplicada a cada elemento do vetor. Para os quatro e-mails de treino:

```
z = X @ w + b
p = 1 / (1 + np.exp(-z))
print("z:", np.round(z, 2))
print("p:", np.round(p, 2))
print("predict_proba:", np.round(modelo.predict_proba(X)[:, 1], 2))
```

```
Saída: z: [-1.18  1.18 -1.18  1.18]
p: [0.24 0.76 0.24 0.76]
predict_proba: [0.24 0.76 0.24 0.76]
```

Aqui `X` não tem a coluna de uns, porque o scikit-learn guarda o viés separado em `intercept_`. Os dois e-mails de spam recebem probabilidade 0,76, e os dois legítimos, 0,24.

A Fronteira de Decisão

Os pontos em que o modelo fica exatamente indeciso são aqueles com $z = 0$. Para o exemplo de spam:

$$0{,}471\, x_1 + 0{,}942\, x_2 - 2{,}1195 = 0 \quad \Longrightarrow \quad x_2 = 2{,}25 - 0{,}5\, x_1$$

Dividindo tudo por 0,942, obtemos a reta que separa as duas regiões da Figura 4.2: ela corta o eixo de "urgente" em 2,25 e desce meia unidade a cada "grátis" a mais. Os e-mails acima da reta têm $z > 0$ e são classificados como spam.

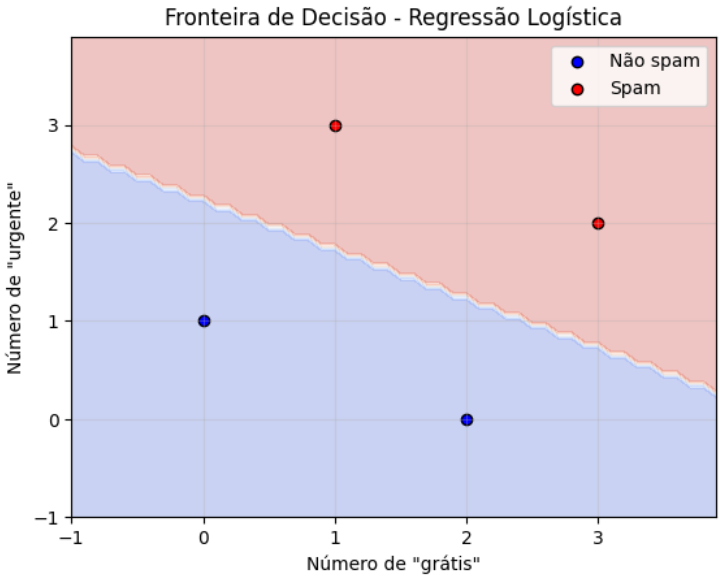


Figura 4.2: Fronteira de decisão da regressão logística no exemplo de spam: a reta $x_2 = 2{,}25 - 0{,}5\,x_1$. A região azul corresponde a "não spam" ($z < 0$) e a vermelha a "spam" ($z > 0$), com os quatro e-mails de treino destacados.

Com d *features*, o conjunto $\mathbf{w}^\top \mathbf{x} + b = 0$ é um hiperplano: uma reta em duas dimensões, um plano em três. Por isso a regressão logística é um **classificador linear**: sua fronteira nunca faz curvas. Classes que exigem uma fronteira curva, como uma classe concentrada no centro e a outra ao redor, não são separadas corretamente sem a criação de novas *features*.

O vetor $\mathbf{w}$ também tem papel geométrico. Ele é perpendicular à fronteira e aponta para o lado da classe 1: aqui, $\mathbf{w} = (0{,}471;\ 0{,}942)$ aponta para mais "grátis" e, sobretudo, mais "urgente". Dividindo o escore pela norma de $\mathbf{w}$, obtemos a distância (com sinal) de um ponto até a fronteira:

$$\text{distância} = \frac{\mathbf{w}^\top \mathbf{x} + b}{\|\mathbf{w}\|}, \qquad \|\mathbf{w}\| = \sqrt{0{,}471^2 + 0{,}942^2} \approx 1{,}053$$

Para o novo e-mail, $-0{,}2355/1{,}053 \approx -0{,}22$: ele está a apenas 0,22 unidades da fronteira, do lado "não spam". Essa proximidade explica uma probabilidade tão perto de 0,5.

Interpretando os Pesos: Chances

Isolando o escore na Equação 4.3, chegamos a uma forma linear:

$$\ln\left(\frac{p}{1-p}\right) = \mathbf{w}^\top \mathbf{x} + b \tag{4.5}$$

onde $p = P(y = 1 \mid \mathbf{x})$. A razão $p/(1-p)$ é a **chance** (*odds*): quantas vezes o spam é mais provável que o não spam. Para o novo e-mail, $0{,}4414/0{,}5586 \approx 0{,}79$, ou 0,79 para 1. O logaritmo da chance (*log-odds*) é exatamente o logit z.

Como o log-odds é linear nas *features*, cada peso tem uma leitura direta. Cada "grátis" a mais soma 0,471 ao log-odds, o que multiplica a chance de spam por $e^{0,471} \approx 1,60$. Cada "urgente" a mais soma 0,942, multiplicando a chance por $e^{0,942} \approx 2,57$. Um peso positivo aumenta a chance da classe 1, um negativo a reduz e um peso zero indica uma *feature* ignorada pelo modelo. Valem os mesmos cuidados da regressão: a leitura supõe as demais *features* constantes, e comparar pesos só faz sentido quando as *features* estão na mesma escala (aqui, ambas são contagens de palavras).

Como os Pesos São Ajustados: Entropia Cruzada

Na regressão linear, o custo era o MSE. Para modelos que produzem probabilidades, o custo padrão é a **entropia cruzada binária** (*log-loss*):

$$J(\mathbf{w}, b) = -\frac{1}{n}\sum_{i=1}^{n}\Big[\, y_i \ln p_i + (1 - y_i)\ln(1 - p_i)\,\Big], \qquad p_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b) \tag{4.6}$$

Como y_i vale 0 ou 1, só um dos dois termos atua em cada exemplo. Se o exemplo é da classe 1, o custo é $-\ln p_i$; se é da classe 0, é $-\ln(1 - p_i)$. Em ambos os casos, o custo é o logaritmo negativo da probabilidade que o modelo deu à classe correta: perto de zero quando essa probabilidade é alta e cada vez maior conforme ela se aproxima de zero. Um erro cometido com confiança custa muito mais que uma dúvida.

No exemplo, os e-mails de spam têm $p_i = 0,76$ (custo $-\ln 0,76 \approx 0,27$) e os legítimos têm $p_i = 0,24$ (custo $-\ln 0,76 \approx 0,27$). A média é $J \approx 0,27$.

Diferente do MSE da regressão linear, a entropia cruzada não tem uma fórmula fechada como a equação normal. Os pesos são encontrados por métodos iterativos da família do gradiente descendente, que partem de um palpite e o melhoram passo a passo (Capítulo 9). O `LogisticRegression` faz isso internamente com o algoritmo `lbfgs`.

Nosso exemplo tem uma particularidade: os quatro e-mails podem ser separados perfeitamente por uma reta. Nesse caso, a entropia cruzada diminui sempre que os pesos crescem, porque as probabilidades se aproximam de 0 e 1, e o mínimo só seria atingido com pesos infinitos. Os pesos ficam finitos porque o `LogisticRegression` aplica, por padrão, uma **regularização** L2: soma ao custo uma penalidade proporcional a $\|\mathbf{w}\|^2$, que desencoraja pesos grandes. O parâmetro `C` controla essa penalidade (padrão `C=1`; quanto maior `C`, mais fraca ela é). A Tabela 4.1 mostra o efeito.

`C`	$\mathbf{w}$	b	$P(\textbf{spam})$ **do novo e-mail**
1	(0,47; 0,94)	−2,12	0,44
10	(1,13; 2,25)	−5,07	0,36
100	(1,87; 3,73)	−8,40	0,28
10.000	(3,00; 6,65)	−14,29	0,16

Tabela 4.1: Pesos da regressão logística no exemplo de spam para diferentes valores de `C`.

A fronteira quase não se move (até `C=100` ela continua sendo $x_2 = 2{,}25 - 0{,}5\,x_1$), mas a sigmoide fica cada vez mais íngreme e as probabilidades mais extremas. A classe prevista para o novo e-mail não muda, e a probabilidade cai de 0,44 para 0,16. As probabilidades da regressão logística são estimativas que dependem da regularização, sobretudo com poucos dados.

4.2.2 K-Nearest Neighbors (KNN)

O algoritmo K-Nearest Neighbors (KNN) é um método simples e intuitivo para classificação. Ele não aprende pesos: guarda os exemplos de treino e, para classificar um novo dado, encontra os k exemplos mais próximos (vizinhos) e atribui a classe mais comum entre eles. "Próximo" é medido, em geral, pela distância euclidiana, que em notação vetorial é a norma da diferença entre dois vetores de *features*:

$$\text{dist}(\mathbf{x}, \mathbf{x}') = \|\mathbf{x} - \mathbf{x}'\| = \sqrt{(\mathbf{x} - \mathbf{x}')^\top (\mathbf{x} - \mathbf{x}')} = \sqrt{\sum_{j=1}^{d} (x_j - x'_j)^2} \tag{4.7}$$

onde $\mathbf{x}$ é o novo exemplo, $\mathbf{x}'$ é um exemplo de treino e $\mathbf{x} - \mathbf{x}'$ é o vetor das diferenças, *feature* a *feature*. É a mesma norma usada na função de custo da regressão (Equação 3.10), agora aplicada à diferença entre dois pontos: o produto escalar desse vetor com ele mesmo soma as diferenças ao quadrado, e a raiz quadrada devolve o comprimento.

Usando o mesmo exemplo de e-mails, suponha que queremos classificar um novo e-mail com 2 "grátis"e 1 "urgente". O KNN calcula a distância desse ponto aos exemplos de treinamento e escolhe, por exemplo, os 3 vizinhos mais próximos ($k = 3$). Se dois forem "spam"e um "não spam", o e-mail é classificado como spam. Veja a implementação em Python:

```python
from sklearn.neighbors import KNeighborsClassifier
import numpy as np

# Dados: [nº de "grátis", nº de "urgente"]
X = np.array([[2, 0], [1, 3], [0, 1], [3, 2]])
y = [0, 1, 0, 1]  # 0 = Não spam, 1 = Spam

# Treina o modelo com k=3
modelo = KNeighborsClassifier(n_neighbors=3)
modelo.fit(X, y)

# Previsão para novo e-mail: 2 "grátis", 1 "urgente"
novo_email = np.array([[2, 1]])
predicao = modelo.predict(novo_email)
print(f"Classe prevista: {'Spam' if predicao[0] == 1 else 'Não spam'}")
```

```
Saída:
    Classe prevista: Não spam
```

A Tabela 4.2 mostra a conta para o novo e-mail $\mathbf{x} = (2;\ 1)$.

Exemplo de treino $\mathbf{x}'$	**Classe**	$\mathbf{x} - \mathbf{x}'$	$\|\mathbf{x} - \mathbf{x}'\|$
$(2;\ 0)$	não spam	$(0;\ 1)$	$\sqrt{0^2 + 1^2} = 1$
$(1;\ 3)$	spam	$(1;\ -2)$	$\sqrt{1^2 + 2^2} = \sqrt{5} \approx 2{,}24$
$(0;\ 1)$	não spam	$(2;\ 0)$	$\sqrt{2^2 + 0^2} = 2$
$(3;\ 2)$	spam	$(-1;\ -1)$	$\sqrt{1^2 + 1^2} = \sqrt{2} \approx 1{,}41$

Tabela 4.2: Distâncias euclidianas do novo e-mail (2 "grátis", 1 "urgente") aos exemplos de treino.

Os três vizinhos mais próximos estão a distâncias 1, 1,41 e 2. Dois deles são "não spam" e um é "spam", então a classe prevista é "não spam". Em NumPy, todas as distâncias saem de uma vez:

```
distancias = np.linalg.norm(X - novo_email, axis=1)
print("Distâncias:", np.round(distancias, 2))
print("Índices dos 3 vizinhos:", np.argsort(distancias)[:3])
```

```
Saída: Distâncias: [1.   2.24 2.   1.41]
Índices dos 3 vizinhos: [0 3 2]
```

A subtração `X - novo_email` é feita linha a linha, e `np.linalg.norm(..., axis=1)` calcula a norma de cada linha. Como a distância soma as diferenças de todas as *features*, uma *feature* com valores grandes domina as demais. Se acrescentássemos o tamanho do e-mail em caracteres, com valores na casa das centenas, as contagens de "grátis" e "urgente" praticamente deixariam de influenciar a escolha dos vizinhos. Por isso o KNN pede *features* padronizadas (Capítulo 2).

4.3 Considerações Finais

Os dois modelos usam a mesma notação de maneiras opostas. A regressão logística resume cada exemplo em um produto escalar $\mathbf{w}^\top \mathbf{x} + b$, traça uma fronteira linear e tem pesos interpretáveis como efeitos sobre a chance da classe positiva. O KNN não aprende parâmetros: decide pela norma da diferença entre vetores, o que permite fronteiras de qualquer formato, mas o torna sensível a outliers, à escala das *features* e ao valor de k, além de lento com conjuntos de treino grandes, já que cada previsão calcula a distância a todos os exemplos guardados. O Capítulo 6 mostra como avaliar esses modelos.

Capítulo 5

Clustering: Agrupamento de Dados

5.1 O que é Aprendizado Não Supervisionado e Clustering?

Diferente do aprendizado supervisionado, onde os dados possuem rótulos que guiam o treinamento, o aprendizado não supervisionado trabalha com dados não rotulados, buscando identificar padrões ou estruturas intrínsecas sem orientação prévia. Seu objetivo é explorar os dados para encontrar relações naturais, como grupos ou anomalias, sem saber de antemão o que essas relações significam. Essa abordagem é amplamente usada em tarefas como segmentação de mercado, compressão de dados e detecção de outliers.

Dentro do aprendizado não supervisionado, o clustering é uma das técnicas mais populares. Clustering consiste em agrupar dados semelhantes em clusters, de modo que os pontos dentro de um mesmo cluster sejam mais parecidos entre si do que com os de outros clusters. Um exemplo prático é a segmentação de clientes em uma loja online: clientes com padrões de compra semelhantes (ex.: frequência e valor gasto) podem ser agrupados para estratégias de marketing direcionadas. Outras aplicações incluem organização de documentos, análise de redes sociais e até astronomia, como agrupar estrelas por características.

5.2 Algoritmo K-Means

O K-Means é um dos algoritmos de clustering mais simples e amplamente utilizados. Ele divide os dados em k grupos, onde k é um número pré-definido pelo usuário, minimizando a variância dentro de cada cluster. O funcionamento do K-Means pode ser descrito nos seguintes passos:

1. **Inicialização**: Escolha k pontos aleatórios como centroides iniciais dos clusters.

2. **Atribuição**: Associe cada ponto de dados ao centroide mais próximo, usando a

distância Euclidiana:

$$\text{dist}(\mathbf{x}, \mathbf{c}) = \|\mathbf{x} - \mathbf{c}\| = \sqrt{\sum_{j=1}^{d} (x_j - c_j)^2} \tag{5.1}$$

onde $\mathbf{x}$ é o vetor de *features* de um ponto, $\mathbf{c}$ é o vetor do centroide e d é o número de *features*. É a mesma distância usada pelo KNN no Capítulo 4.

3. **Atualização**: Recalcule os centroides como a média dos pontos atribuídos a cada cluster.

4. **Iteração**: Repita os passos 2 e 3 até que os centroides não mudem significativamente ou um número máximo de iterações seja atingido.

O resultado é um conjunto de k clusters, onde cada ponto pertence ao cluster cujo centroide está mais próximo. Apesar de sua simplicidade, o K-Means tem limitações: ele assume clusters esféricos e pode ser sensível à escolha inicial dos centroides e ao valor de k.

5.3 Exemplo Prático: Segmentar Clientes por Compras

Considere uma loja online que deseja segmentar clientes com base em duas características: o número de compras mensais e o valor médio gasto por compra (em reais). Um conjunto de dados fictício com 6 clientes poderia ser:

- Cliente 1: 2 compras, R$50,
- Cliente 2: 5 compras, R$120,
- Cliente 3: 1 compra, R$30,
- Cliente 4: 3 compras, R$60,
- Cliente 5: 6 compras, R$150,
- Cliente 6: 4 compras, R$100.

Vamos usar o K-Means com $k = 2$ para agrupar esses clientes em dois clusters: "compradores ocasionais"e "compradores frequentes". A seguir, mostramos como implementar isso em Python com a biblioteca scikit-learn:

```
import numpy as np
from sklearn.cluster import KMeans

# Dados: [nº de compras, valor médio (R£)]
X = np.array([[2, 50], [5, 120], [1, 30], [3, 60], [6, 150], [4, 100]])

# Aplica K-Means com k=2
kmeans = KMeans(n_clusters=2, random_state=42)
kmeans.fit(X)

# Resultados
clusters = kmeans.labels_
centroides = kmeans.cluster_centers_
print("Atribuição dos clusters:", clusters)
print("Centroides:", centroides)
```

A saída é:

```
Atribuição dos clusters: [0 1 0 0 1 1]
Centroides: [[  2.          46.66666667]
 [  5.         123.33333333]]
```

Isso indica que os clientes 1, 3 e 4 (cluster 0) são "compradores ocasionais"(média de 2 compras, R$46,67), enquanto os clientes 2, 5 e 6 (cluster 1) são "compradores frequentes"(média de 5 compras, R$123,33).

5.4 Visualização dos Clusters

Para entender melhor os resultados, podemos visualizar os clusters em um gráfico de dispersão. O código Python a seguir gera essa visualização, destacando os pontos dos clientes e os centroides:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Dados
X = np.array([[2, 50], [5, 120], [1, 30], [3, 60], [6, 150], [4, 100]])

# Aplica K-Means
kmeans = KMeans(n_clusters=2, random_state=42)
kmeans.fit(X)
```

```
# Plota os clusters
plt.scatter(X[:, 0], X[:, 1], c=kmeans.labels_,
                                cmap='viridis',
                                edgecolors='k',
                                label='Clientes')
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
            c='red', marker='x', s=200, label='Centroides')
plt.xlabel('Número de Compras Mensais')
plt.ylabel('Valor Médio por Compra (R$)')
plt.title('Segmentação de Clientes com K-Means')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('clusters_clientes.png', dpi=300, bbox_inches='tight')
plt.show()
```

A Figura 5.1 mostra o resultado, com os clientes em cores diferentes por cluster e os centroides marcados em vermelho.

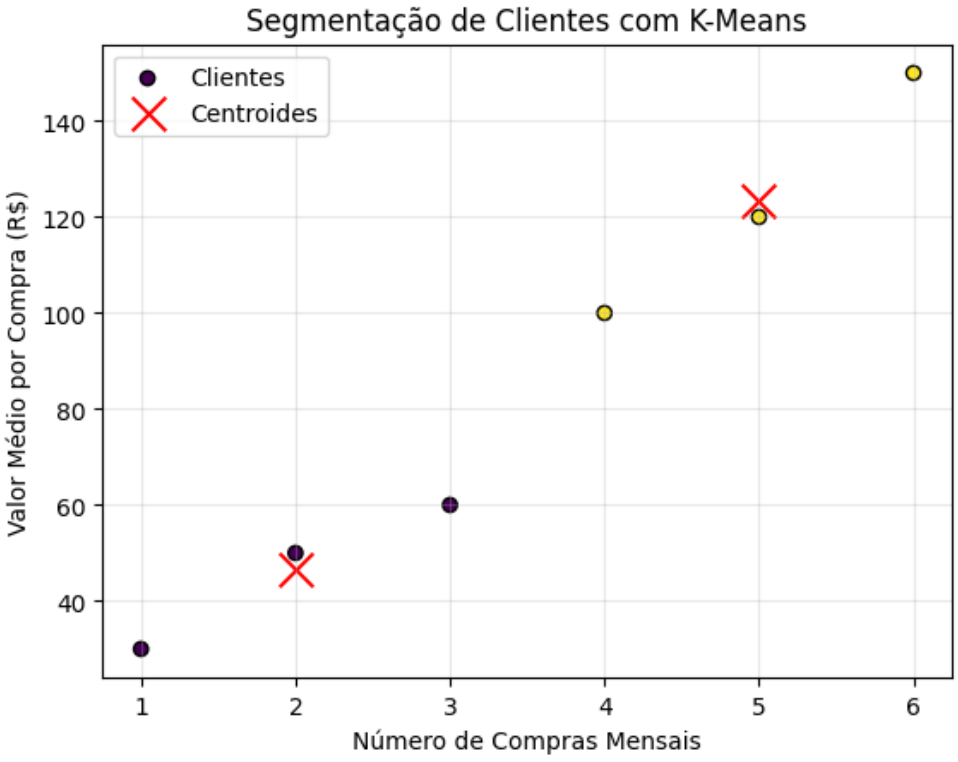


Figura 5.1: Visualização dos clusters de clientes com K-Means ($k = 2$), destacando compradores ocasionais e frequentes.

5.5 Interpretação e Limitações

O gráfico revela dois grupos distintos: compradores ocasionais com poucas compras e gastos baixos, e compradores frequentes com mais compras e gastos maiores. Essa segmentação pode orientar estratégias de marketing, como oferecer descontos aos ocasionais ou programas de fidelidade aos frequentes. No entanto, o K-Means exige que k seja escolhido previamente, o que nem sempre é trivial. Técnicas como o método do cotovelo (elbow method) podem ajudar a determinar o k ideal, um tópico que pode ser explorado em estudos mais avançados.

Outro cuidado: as duas *features* estão em escalas diferentes (1 a 6 compras contra R$30 a R$150), então a distância Euclidiana é dominada pelo valor gasto. Aqui o resultado é o mesmo porque as duas variáveis crescem juntas, mas em um projeto real o correto é padronizar os dados antes de aplicar o K-Means, como visto no Capítulo 2.

Capítulo 6

Avaliação e Melhoria de Modelos

6.1 Overfitting e Underfitting: Definições e Causas

A construção de modelos de Machine Learning envolve encontrar um equilíbrio entre aprender os padrões dos dados e generalizar para novos exemplos. Dois problemas comuns nesse processo são o *overfitting* e o *underfitting*. O *overfitting* ocorre quando um modelo se ajusta excessivamente aos dados de treinamento, capturando inclusive ruídos ou detalhes irrelevantes, o que prejudica seu desempenho em dados novos. Por outro lado, o *underfitting* acontece quando o modelo é simples demais, falhando em aprender os padrões principais dos dados.

As causas do *overfitting* incluem modelos muito complexos (ex.: muitas variáveis ou camadas), poucos dados de treinamento ou falta de regularização. O sintoma típico é um bom desempenho nos dados de treinamento e um desempenho ruim em dados novos. Já o *underfitting* pode resultar de modelos simples (ex.: regressão linear para dados não lineares) ou treinamento insuficiente; aumentar a capacidade do modelo (mais parâmetros ou *features*) é uma das soluções. Entender esses conceitos é essencial para avaliar e melhorar o desempenho de um modelo, garantindo que ele seja útil em aplicações reais.

6.2 Divisão Treino/Teste e Validação Cruzada

Para avaliar a capacidade de generalização de um modelo, é prática padrão dividir os dados em dois conjuntos: *treino* e *teste*. O conjunto de treino é usado para ajustar o modelo, enquanto o conjunto de teste, que contém dados não vistos durante o treinamento, serve para medir seu desempenho real. Uma divisão comum é 80% para treino e 20% para teste, mas isso pode variar conforme o tamanho do dataset.

Quando os dados são limitados, a divisão simples pode ser instável, pois o resultado depende de quais dados caem no teste. Nesses casos, a validação cruzada é uma alternativa robusta. Na validação cruzada de k dobras (*k-fold cross-validation*), os dados são divididos em k partes iguais. O modelo é treinado k vezes, usando $k-1$ partes para treino e a parte restante para teste em cada iteração. A média dos desempenhos fornece uma estimativa

mais confiável. Por exemplo, com $k = 5$, cada dobra representa 20% dos dados como teste.

6.3 Exemplo Prático: Avaliar um Modelo de Regressão

Considere um conjunto de dados fictício para prever o preço de casas (em milhares de reais) com base em sua área (em m^2). Os dados são:

- Casa 1: 50 m^2, R$200 mil,
- Casa 2: 70 m^2, R$280 mil,
- Casa 3: 60 m^2, R$240 mil,
- Casa 4: 80 m^2, R$320 mil,
- Casa 5: 55 m^2, R$210 mil.

Vamos usar uma regressão linear e avaliar seu desempenho com divisão treino/teste:

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Dados: área (m²) e preço (R£ mil)
X = np.array([[50], [70], [60], [80], [55]])  # Característica
y = np.array([200, 280, 240, 320, 210])       # Alvo

# Divide em treino (80%) e teste (20%)
X_treino, X_teste, y_treino, y_teste = train_test_split(X,
                                        y,
                                        test_size=0.2,
                                        random_state=42)

# Treina o modelo
modelo = LinearRegression()
modelo.fit(X_treino, y_treino)

# Previsão no conjunto de teste
y_pred = modelo.predict(X_teste)

# Calcula o erro quadrático médio (MSE)
mse = mean_squared_error(y_teste, y_pred)
print(f"Erro Quadrático Médio (MSE): {mse:.2f} (R$ mil)^2")
```

```
Saída: Erro Quadrático Médio (MSE): 2.09 (R$ mil)^2
```

Com `random_state=42`, o teste ficou com a Casa 2 (70 m², R$280 mil). O modelo previu R$278,55 mil, um erro de R$1,45 mil, que ao quadrado resulta no MSE de 2,09. Um único exemplo de teste diz pouco sobre o desempenho; para uma avaliação mais robusta, usamos validação cruzada:

```python
from sklearn.model_selection import cross_val_score

# Validação cruzada com 5 dobras
scores = cross_val_score(modelo, X, y, cv=5, scoring='neg_mean_squared_error')
mse_cv = -scores.mean()
print(f"MSE médio com validação cruzada: {mse_cv:.2f} (R$ mil)^2")
```

```
Saída: MSE médio com validação cruzada: 33.70 (R$ mil)^2
```

O MSE médio mostra como o modelo se sai em média ao prever dados novos, considerando várias divisões diferentes dos dados. Como temos apenas 5 casas, `cv=5` deixa uma casa de fora por vez (*leave-one-out*). O valor de 33,70 é bem maior que os 2,09 da divisão única, sinal de que aquela divisão foi otimista.

6.4 Métricas de Avaliação

Para tarefas de classificação, métricas específicas ajudam a avaliar o desempenho além do erro bruto. A Tabela 6.1 compara três métricas comuns:

Métrica	**Descrição**
Accuracy	Proporção de previsões corretas (acertos / total). Ideal para classes balanceadas.
Precision	Proporção de previsões positivas corretas (verdadeiros positivos / (verdadeiros positivos + falsos positivos)). Útil quando falsos positivos são custosos.
Recall	Proporção de positivos reais identificados (verdadeiros positivos / (verdadeiros positivos + falsos negativos)). Importante quando falsos negativos são críticos.

Tabela 6.1: Comparação de métricas de avaliação para classificação.

Por exemplo, em um modelo de detecção de spam, alta precision reduz falsos positivos (e-mails legítimos marcados como spam), enquanto alto recall minimiza falsos negativos (spam não detectado).

6.5 Matriz de Confusão

A avaliação de modelos de aprendizado de máquina exige mais do que métricas agregadas como acurácia, especialmente em tarefas de classificação onde os erros têm impactos distintos. A matriz de confusão é uma ferramenta fundamental que detalha o desempenho de um modelo ao comparar previsões com valores reais, oferecendo uma análise granular dos acertos e erros por classe. Isso é essencial em problemas binários (ex.: fraude vs. não fraude) ou multiclasse, permitindo identificar padrões de falhas que métricas simples podem ocultar.

Em uma classificação binária, a matriz de confusão é uma tabela 2x2 composta por:

- **Verdadeiros Positivos (VP ou TP)**: Casos positivos (ex.: fraudes) corretamente previstos como positivos.
- **Verdadeiros Negativos (VN ou TN)**: Casos negativos (ex.: não fraudes) corretamente previstos como negativos.
- **Falsos Positivos (FP)**: Casos negativos previstos incorretamente como positivos (erro tipo I).
- **Falsos Negativos (FN)**: Casos positivos previstos incorretamente como negativos (erro tipo II).

Esses elementos permitem calcular métricas derivadas, como precisão (Precisão $= \frac{VP}{VP+FP}$), revocação ou *recall* (Revocação $= \frac{VP}{VP+FN}$) e F1-score ($F1 = 2 \cdot \frac{\text{Precisão} \cdot \text{Revocação}}{\text{Precisão} + \text{Revocação}}$), que balanceiam os custos de diferentes tipos de erros.

A matriz de confusão é indispensável em *Machine Learning* para avaliar o desempenho em cenários práticos. Por exemplo, em detecção de fraudes em cartões de crédito, falsos negativos (fraudes não detectadas) podem custar caro financeiramente, enquanto falsos positivos (transações legítimas bloqueadas) afetam a experiência do cliente. Integrada às técnicas de divisão de treino/teste e validação cruzada já discutidas neste capítulo, ela fornece uma visão mais rica para otimizar modelos e ajustar estratégias.

6.5.1 Exemplo Prático: Detecção de Fraudes em Cartões de Crédito

Vamos usar o dataset *Credit Card Fraud Detection* do Kaggle (https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud), que contém transações de cartões de crédito europeus em setembro de 2013, com 284.807 registros e 31 colunas (tempo da transação,

28 variáveis anonimizadas, valor da transação e rótulo de fraude: 0 para não fraude, 1 para fraude). O dataset é altamente desbalanceado, com apenas 492 fraudes (0,17% do total). Treinaremos um modelo de regressão logística e analisaremos sua matriz de confusão. O código Python a seguir implementa esse processo:

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt

# Carregando o dataset (assumindo que foi baixado do Kaggle)
df = pd.read_csv('creditcard.csv')
X = df.drop(columns=['Class'])  # Features (excluindo o rótulo)
y = df['Class']  # Rótulo (0 = não fraude, 1 = fraude)

# Dividindo em treino e teste (20% para teste)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42,
stratify=y)

# Treinando o modelo de regressão logística com pesos balanceados
modelo = LogisticRegression(max_iter=1000, class_weight='balanced',
random_state=42)
modelo.fit(X_train, y_train)

# Fazendo previsões no conjunto de teste
y_pred = modelo.predict(X_test)

# Calculando a matriz de confusão
matriz = confusion_matrix(y_test, y_pred)
print("Matriz de Confusão:")
print(matriz)

# Relatório detalhado com precisão, revocação e F1-score
print("\nRelatório de Classificação:")
print(classification_report(y_test, y_pred, target_names=['Não Fraude', 'Fraude']))

# Visualização da matriz de confusão
plt.figure(figsize=(6, 4))
sns.heatmap(matriz, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Não Fraude', 'Fraude'], yticklabels=['Não Fraude',
```

```
            'Fraude'])
plt.xlabel('Previsto')
plt.ylabel('Real')
plt.title('Matriz de Confusão - Detecção de Fraudes')
plt.show()
```

Resultados: No conjunto de teste com 56.962 amostras (20% do total), os resultados foram:

- Matriz de Confusão:

	Previsto Não Fraude	Previsto Fraude
Real Não Fraude	55313 (VN)	1551 (FP)
Real Fraude	8 (FN)	90 (VP)

- Relatório de Classificação:

	precision	recall	f1-score	support
Não Fraude	1.00	0.97	0.99	56864
Fraude	0.05	0.92	0.10	98
accuracy			0.97	56962
macro avg	0.53	0.95	0.54	56962
weighted avg	1.00	0.97	0.98	56962

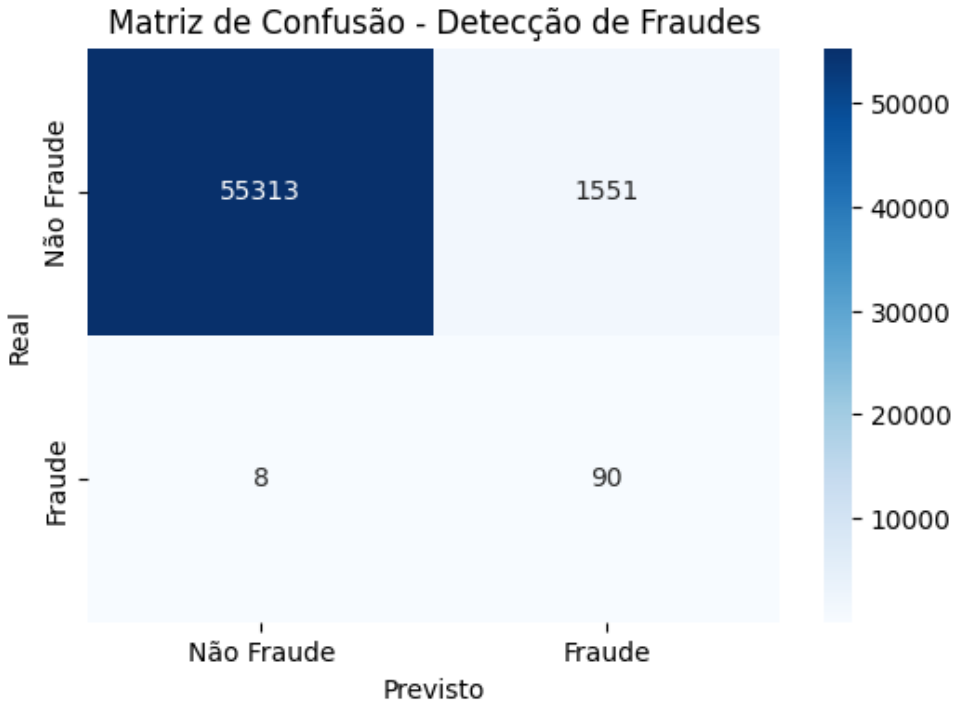


Figura 6.1: Matriz de confusão do modelo de detecção de fraudes: o *heatmap* destaca os acertos na diagonal e os erros fora dela.

6.5.2 Interpretação dos Resultados

A matriz de confusão (Figura 6.1) mostra que o modelo identificou corretamente 55313 transações como não fraudulentas (VN) e 90 como fraudulentas (VP), totalizando 55403 acertos em 56962 amostras, resultando em uma acurácia de 97%. No entanto, a alta acurácia é enganosa devido ao desbalanceamento: as não fraudes dominam o dataset. Os 1551 falsos positivos (FP) são transações legítimas marcadas como fraudes, enquanto os 8 falsos negativos (FN) são fraudes não detectadas.

O relatório detalha o desempenho por classe. Para "Não Fraude", a precisão é praticamente 1,00 e a revocação é 0,97, refletindo a facilidade de prever a classe majoritária. Para "Fraude", a revocação de 0,92 mostra que 90 das 98 fraudes reais foram capturadas, um bom resultado em um dataset tão desbalanceado. A precisão, porém, é de apenas 0,05: de cada 100 alertas de fraude, cerca de 95 eram transações legítimas (1551 falsos positivos contra 90 acertos). O F1-score de 0,10 reflete esse desequilíbrio, pois a média harmônica fica próxima do menor dos dois valores. Um cuidado ao ler o relatório: a linha *macro avg* (0,53 e 0,95) é a média simples das duas classes e não descreve o desempenho na classe "Fraude".

Em um contexto real, como bancos ou empresas de cartão, as 8 FN (fraudes perdidas) representam perdas financeiras, enquanto os 1551 FP podem irritar clientes com bloqueios desnecessários. A alta revocação é positiva para minimizar fraudes, mas a precisão muito

baixa sugere que ajustes, como alterar o limiar de decisão do modelo ou usar técnicas de reamostragem (ex.: SMOTE), poderiam reduzir FP. A matriz de confusão, combinada com validação cruzada, expõe essas nuances, permitindo refinar o modelo para priorizar o que importa: detectar fraudes sem exagerar nos falsos alarmes. Esse insight é fundamental para aplicações práticas de *Machine Learning*.

6.6 Interpretação e Próximos Passos

No exemplo de regressão, o MSE baixo da divisão única parecia indicar boa generalização, mas a validação cruzada ofereceu uma visão mais confiável. Para melhorar modelos, técnicas como regularização (ex.: Ridge, Lasso) podem reduzir *overfitting*, enquanto mais dados ou modelos complexos combatem *underfitting*. No caso de classificação, escolher a métrica certa depende do problema: *accuracy* para balanceamento geral, *precision* ou *recall* para prioridades específicas. Esses tópicos podem ser aprofundados em estudos avançados.

Capítulo 7

Aplicando o Aprendizado em Dados Reais

7.1 Ferramentas e Preparo

Antes de trabalhar com os dados reais e construir nosso classificador, é importante preparar o ambiente de desenvolvimento. Esta seção apresenta brevemente as ferramentas essenciais para este capítulo, ideais para iniciantes que ainda não estão familiarizados com elas.

7.1.1 Python e sua Versão Atual

Python é uma linguagem de programação amplamente usada em Machine Learning devido à sua simplicidade e vasta comunidade. Recomenda-se usar uma versão recente do Python 3, que oferece melhor desempenho e suporte às bibliotecas modernas. Para instalar, visite `python.org` e siga as instruções para o seu sistema operacional (Windows, Mac ou Linux). Todas as ferramentas a seguir assumem que o Python está instalado.

7.1.2 Google Colab e Jupyter Notebook

Para executar os códigos deste capítulo, recomendamos a utilização do Google Colab, no entanto você pode usar o Jupyter Notebook ou alguma outra IDE de sua preferência. O Google Colab é uma plataforma online gratuita que roda Python diretamente no navegador, sem necessidade de instalação local — basta uma conta Google e acesso à internet. Ele é perfeito para iniciantes, pois já vem com bibliotecas como scikit-learn pré-instaladas. Alternativamente, o Jupyter Notebook é um ambiente local que permite escrever e executar código em blocos interativos.

7.1.3 Scikit-learn

O scikit-learn é uma biblioteca de Machine Learning em Python, conhecida por sua facilidade de uso e ampla gama de algoritmos, como regressão, classificação e clustering. No Google Colab ela já vem instalada; em um ambiente local, instale com `pip install scikit-learn`. Com essas ferramentas, estamos prontos para trabalhar com os dados de notícias.

7.2 Classificando Notícias com Aprendizado de Máquina

Neste capítulo, vamos usar aprendizado de máquina pra classificar notícias como **positivas** ou **negativas**. Temos um dataset pequeno, com 845 notícias coletadas entre março e abril de 2025, das quais 200 já foram classificadas manualmente na coluna `sentimento`. Não é uma quantidade ideal para um modelo de alto desempenho, mas é suficiente para servir de exemplo. Treinaremos um classificador `SVC`, usando o `TfidfVectorizer` para a vetorização das palavras, e veremos até onde ele chega.

Um aviso sobre os rótulos: eles refletem a leitura do fato noticiado (uma boa ou uma má notícia), e não o tom emocional das palavras. "Inflação cai"foi marcada como positiva; "preço do café sobe", como negativa. É uma classificação subjetiva, e outra pessoa poderia discordar em vários casos.

7.2.1 Conectando ao Google Drive

Primeiro, faremos a conexão do Google Colab com o Drive:

```
from google.colab import drive
drive.mount('/content/drive')
```

O Colab pedirá permissão para acessar o Drive.

7.2.2 Download do arquivo para testes

Os comandos `!wget` e `!mv` do Google Colab fazem, respectivamente, o download do arquivo no GitHub e a movimentação dele para a raiz do seu Drive, já conectado anteriormente.

```
import os

def download_and_save(url, root_drive):
    file_name = os.path.basename(url)
    !wget {url} -O {file_name}
    !mv {file_name} {root_drive}
    print(f"Arquivo '{file_name}' salvo no Drive em '{root_drive}'.")
```

```
url_github = "https://raw.githubusercontent.com/ronaldandrade/ebook/refs/
↪  heads/master/noticias.csv"
root_drive = "/content/drive/MyDrive/noticias.csv"

download_and_save(url_github, root_drive)
```

Sempre que a função `download_and_save()` for chamada, ela fará o download do arquivo `noticias.csv` novamente.

7.2.3 Carregando o Dataset

Utilizando a biblioteca `pandas` para carregar o dataset e com o comando `print()` você já conseguirá ver listado os primeiros títulos:

```
import pandas as pd
caminho = '/content/drive/MyDrive/noticias.csv'
df = pd.read_csv(caminho)
titulos = df['titulo'].tolist()
print("Primeiros 5 títulos:", titulos[:5])
```

7.2.4 Analisando o Dataset

Antes de construir qualquer modelo de aprendizado de máquina, é essencial entender o conjunto de dados com o qual estamos trabalhando. A pré-visualização do dataset permite identificar a estrutura, os tipos de variáveis e possíveis problemas, como valores ausentes ou inconsistências. Em Python, as funções `head()` e `info()` da biblioteca `pandas` são ferramentas simples e excelentes para essa tarefa, oferecendo uma visão rápida dos dados e suas características técnicas.

A função `df.head()` exibe as primeiras cinco linhas do `DataFrame`, dando uma amostra direta do conteúdo. Já `df.info()` fornece um resumo técnico: número de entradas, colunas, tipos de dados (ex.: `int`, `object`) e a presença de valores nulos. Juntas, elas formam o ponto de partida para decidir os próximos passos, como limpeza ou transformação dos dados.

```
print(df.head())  # Exibe as primeiras 5 linhas
```

A saída de `df.head()` mostra uma tabela como a seguir (com URLs e conteúdos simplificados para clareza):

Tabela 7.1: Amostra do Dataset de Notícias (df.head())

id	titulo	conteudo	url	data_publicacao	resumo	sentimento
1	Inflação argen...	¡img=”...≫	https://g1....	2025-03-15 00:50:10	NaN	Positivo
2	Lula diz que ...	¡img=”...≫	https://g1....	2025-03-14 15:09:14	NaN	Positivo
3	Argentina e...	¡img=”...≫	https://g1....	2025-03-14 14:18:25	NaN	Negativo
4	Produção de ve...	¡img=”...≫	https://g1....	2025-03-14 14:00:19	NaN	Positivo
5	Preço do ca...	¡img=”...≫	https://g1....	2025-03-14 13:40:05	NaN	Negativo

A função `head()` traz as 5 primeiras linhas da tabela, nos mostrando suas sete colunas: `id` (identificador numérico), `titulo` (texto), `conteudo` (HTML com imagens), `url` (endereço web), `data_publicacao` (timestamp), `resumo` (valores ausentes) e `sentimento` (categórico: Positivo ou Negativo).

```
print(df.info())  # Resumo técnico do dataset
```

Saída:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 845 entries, 0 to 844
Data columns (total 7 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   id               845 non-null    int64
 1   titulo           845 non-null    object
 2   conteudo         845 non-null    object
 3   url              845 non-null    object
 4   data_publicacao  845 non-null    object
 5   resumo           230 non-null    object
 6   sentimento       200 non-null    object
dtypes: int64(1), object(6)
memory usage: 46.3+ KB
```

O `df.info()` mostra os tipos de dados, o total de linhas e colunas e quantos valores não nulos há em cada coluna. A coluna `resumo`, por exemplo, tem 230 valores não nulos, ou seja, 615 linhas vazias. Já `titulo`, completa nas 845 linhas, e `sentimento`, preenchida em 200, são as colunas essenciais para a nossa tarefa de classificação. Essa análise inicial orienta a limpeza (ex.: descartar `resumo`) e o pré-processamento (ex.: tratar `titulo` como entrada principal), passos que detalharemos adiante.

7.2.5 Preparando os Dados Classificados

Para que o computador entenda como as classificações foram feitas, precisamos passar para uma forma que ele saiba interpretar, ou seja, números. Selecionamos as notícias que já têm rótulo, filtrando pelos valores não nulos da coluna `sentimento` (mais seguro do que confiar na posição das linhas), e convertemos os rótulos para 1 se positivo e 0 se negativo:

```
df_rotulado = df[df['sentimento'].notna()].copy()
titulos_rotulados = df_rotulado['titulo'].tolist()
rotulos = [1 if s == 'Positivo' else 0 for s in df_rotulado['sentimento']]
print("Total de exemplos rotulados:", len(rotulos))
print("Primeiros 5 rótulos:", rotulos[:5])
print("Total de positivos:", sum(rotulos), "Total de negativos:",
↪  len(rotulos) - sum(rotulos))
```

```
Saída: Total de exemplos rotulados: 200
Primeiros 5 rótulos: [1, 1, 0, 1, 0]
Total de positivos: 108 Total de negativos: 92
```

As classes estão razoavelmente equilibradas: 108 notícias positivas e 92 negativas.

7.2.6 Transformando Texto em Números

Agora precisamos fazer o mesmo com os títulos. Usamos o `TfidfVectorizer`, classe do scikit-learn que transforma textos em números:

```
from sklearn.feature_extraction.text import TfidfVectorizer
stopwords_pt = ['e', 'em', 'de', 'para', 'com', 'que', 'do', 'da', 'no',
↪  'na', 'o', 'a', 'as', 'os']
vetorizador = TfidfVectorizer(max_features=1000, stop_words=stopwords_pt,
↪  lowercase=True)
```

Os modelos de Machine Learning não processam textos diretamente, da maneira como nós, humanos, fazemos. Computadores operam com números e cálculos matemáticos, então, para que um modelo possa trabalhar com dados textuais, é necessário convertê-los em uma representação numérica.

No caso do `TfidfVectorizer`, cada título vira uma linha de uma matriz, com uma coluna para cada palavra do vocabulário. O valor de cada célula fica entre 0 e 1: zero quando a palavra não aparece no título, e um peso tanto maior quanto mais a palavra aparece naquele título e mais rara ela é no restante do corpus. Dessa forma, o algoritmo pode analisar padrões estatísticos nos textos e tomar decisões com base nesses números.

Repare que apenas criamos o vetorizador, sem aplicá-lo aos dados. O vocabulário e os pesos devem ser aprendidos somente com os títulos de treino. Se o vetorizador enxergasse também os títulos de teste, o modelo receberia informação de dados que deveria desconhecer, um problema chamado **vazamento de dados** (*data leakage*), que torna a avaliação otimista demais.

7.2.7 Treinando e Testando o Modelo

Os próximos passos consistem no seguinte: dividimos os dados rotulados em duas partes, treino e teste, treinamos um modelo e avaliamos o desempenho.

A função `train_test_split` do scikit-learn separa 80% dos títulos para treino e 20% para teste (`test_size=0.2`). O `random_state` garante que a separação aconteça sempre da mesma maneira, evitando variações nos resultados cada vez que rodamos o código:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(titulos_rotulados,
                                                    rotulos,
                                                    test_size=0.2,
                                                    random_state=42)
print("Treino:", len(X_train), "Teste:", len(X_test))
```

```
Saída: Treino: 160 Teste: 40
```

Ficamos com 160 títulos para treino e 40 para teste. É pouco: quanto maior o dataset, melhor tende a ser o aprendizado e mais confiável fica a medida de acurácia.

7.2.8 Hora do Treinamento

O SVM (Support Vector Machine) é um modelo de aprendizado de máquina muito eficiente para tarefas de classificação, especialmente quando há duas categorias bem definidas. Ele funciona encontrando uma linha (ou hiperplano) que separa os dados da melhor forma possível, maximizando a distância entre as classes.

No código abaixo, juntamos o vetorizador e um SVM com kernel linear em um `Pipeline`. Ao chamar `fit`, o pipeline ajusta o TF-IDF e o SVM usando apenas os dados de treino; ao chamar `predict`, aplica a mesma transformação aos textos novos antes de classificá-los. Depois, avaliamos o desempenho usando acurácia, precisão e recall.

```
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score

modelo = Pipeline([
    ('tfidf', vetorizador),
    ('svm', SVC(kernel='linear', class_weight='balanced'))
])
modelo.fit(X_train, y_train)
y_pred = modelo.predict(X_test)

print("Tamanho do vocabulário:", len(vetorizador.vocabulary_))
print("Acurácia no teste:", accuracy_score(y_test, y_pred))
print("Precisão:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
```

```
Saída: Tamanho do vocabulário: 976
Acurácia no teste: 0.6
Precisão: 0.5833333333333334
Recall: 0.7
```

Os 160 títulos de treino têm 976 palavras distintas, abaixo do limite de 1.000 definido em `max_features`. O modelo acertou 24 dos 40 títulos de teste (acurácia de 0,60). O teste tem 20 notícias positivas e 20 negativas: o modelo reconheceu 14 das 20 positivas (*recall* de 0,70), mas também classificou como positivas 10 das 20 negativas, daí a precisão de 0,58.

É um resultado modesto, pouco acima dos 50% que um chute alcançaria. A principal explicação é a quantidade de dados: com 160 exemplos, a maioria das palavras aparece uma ou duas vezes, e o modelo tem pouco material para associar termos a sentimentos. Além disso, manchetes são textos curtos e o rótulo é subjetivo. Para melhorar o desempenho, podemos rotular mais exemplos, usar uma lista de *stopwords* mais completa, incluir pares de palavras (`ngram_range=(1, 2)`), ajustar hiperparâmetros ou testar outros modelos.

7.2.9 Testando com Novas Notícias

Vamos testar o modelo com duas notícias fictícias pra ver como ele se comporta:

```
novos_titulos = ["Dólar cai e inflação diminui",
            "EUA tem maior número de deportações nos últimos anos"]
predicoes = modelo.predict(novos_titulos)  # Vetoriza e faz as previsões
print("Previsões novas:", ["Positivo" if p == 1 else "Negativo" for p in
↪  predicoes])
```

```
Saída: Previsões novas: ['Positivo', 'Positivo']
```

O título "Dólar cai e inflação diminui"foi classificado como positivo, o que parece razoável. Já "EUA tem maior número de deportações nos últimos anos"também saiu como positivo, uma classificação que muitos leitores contestariam. O motivo está no vocabulário: as palavras "deportações"e "número"não aparecem em nenhum título de treino, então o modelo as ignora e decide com base em termos genéricos como "EUA", "maior"e "anos", por uma margem mínima. As máquinas aprendem padrões estatísticos das palavras que já viram e desconhecem o significado do fato noticiado.

Vale ressaltar também que a interpretação de uma notícia pode variar de pessoa para pessoa. A classificação de sentimentos em textos depende de padrões objetivos e também da percepção individual do leitor, de seu contexto e de suas opiniões.

7.2.10 (Opcional) Classificando Todas as Notícias

Podemos usar o modelo pra prever o sentimento das 645 notícias que ainda não têm rótulo, preservando as 200 classificações manuais:

```
sem_rotulo = df['sentimento'].isna()
predicoes_restantes = modelo.predict(df.loc[sem_rotulo, 'titulo'])
df.loc[sem_rotulo, 'sentimento'] = ['Positivo' if p == 1 else 'Negativo'
↪  for p in predicoes_restantes]
df.to_csv('/content/drive/MyDrive/noticias_classificadas.csv',
↪  index=False)
```

Dado o desempenho medido no teste, esses rótulos automáticos devem ser tratados como um rascunho a revisar. Com isso, você percorreu o fluxo completo: carregar um dataset, preparar os rótulos, vetorizar o texto, treinar, avaliar e aplicar um modelo de classificação.

7.3 Síntese Conceitual das Técnicas Utilizadas

A análise de sentimentos, como explorada ao longo deste capítulo, representa uma interseção poderosa entre ciência de dados, aprendizado de máquina e processamento de linguagem natural, oferecendo ferramentas para interpretar e quantificar emoções em textos. Este capítulo teve como objetivo principal introduzir os leitores a um pipeline prático e replicável para classificar notícias como positivas ou negativas, utilizando um conjunto de dados real e técnicas amplamente reconhecidas no campo. A escolha do ambiente Google Colab proporcionou um espaço acessível e colaborativo para a execução do projeto, enquanto o dataset de notícias — com uma parte de exemplos rotulados manualmente — serviu como base para demonstrar cada etapa do processo, desde a aquisição dos dados até a avaliação e aplicação do modelo treinado.

O crescente volume de informações textuais geradas diariamente, seja em notícias, redes sociais ou outras plataformas digitais, torna a análise de sentimentos uma competência cada vez mais relevante. Empresas utilizam essas técnicas para monitorar a percepção pública sobre produtos e serviços, enquanto pesquisadores as aplicam para compreender tendências sociais e comportamentais. Neste contexto, o conteúdo desta primeira parte do livro buscou não apenas ensinar um método específico, mas também estabelecer fundamentos que permitam aos leitores adaptar e expandir o que foi aprendido para diferentes cenários. A escolha de ferramentas como `pandas` para manipulação de dados, `TfidfVectorizer` para representação textual e o algoritmo Support Vector Machine (SVM) reflete um equilíbrio entre simplicidade e eficácia, adequado tanto para iniciantes quanto para aqueles que desejam aprofundar seus estudos. Além disso, o processo aqui apresentado destaca a importância de etapas críticas como o pré-processamento de dados e a validação de modelos, que são essenciais para garantir resultados confiáveis em projetos de aprendizado de máquina. Ao classificar automaticamente as notícias sem rótulo e testar o modelo com exemplos fictícios, foi possível ilustrar como os conceitos teóricos se traduzem em aplicações práticas. Os parágrafos a seguir consolidam esses aprendizados em uma síntese das técnicas empregadas. Para a representação numérica dos títulos das notícias, empregou-se o método TF-IDF (*Term Frequency-Inverse Document Frequency*), implementado por meio do `TfidfVectorizer` da biblioteca `scikit-learn`. Esse método

transformou os textos em vetores numéricos, eliminando palavras irrelevantes (*stopwords*) e limitando o vocabulário a, no máximo, 1.000 termos. O algoritmo de classificação escolhido foi o Support Vector Machine (SVM) com kernel linear, configurado com pesos balanceados (`class_weight='balanced'`) para compensar a pequena diferença entre as classes positiva (108 exemplos) e negativa (92). O vetorizador e o classificador foram reunidos em um `Pipeline`, evitando vazamento de dados do teste para o treino. O conjunto de notícias rotuladas foi dividido em 80% para treinamento e 20% para teste, utilizando a função `train_test_split`, e o desempenho foi avaliado por acurácia (0,60), precisão (0,58) e *recall* (0,70), resultados modestos e esperados para um treino com apenas 160 exemplos. O modelo treinado foi aplicado tanto para prever o sentimento de notícias fictícias quanto para classificar automaticamente as 645 notícias sem rótulo do dataset, cujos resultados foram salvos em um novo arquivo CSV. Esse fluxo — desde a obtenção dos dados até a aplicação prática do modelo — exemplifica a integração de pré-processamento de texto, aprendizado supervisionado e validação de resultados em um contexto real.

7.4 Para ir mais além

Para aqueles que desejam expandir os conhecimentos adquiridos, aprofundar-se em algum dos assuntos abordados neste livro ou mesmo aperfeiçoar o modelo reproduzido, deixo aqui algumas recomendações de conteúdos mais avançados:

1. **Exploração de outros algoritmos**: Além do SVM, modelos como Random Forest (`RandomForestClassifier`) ou redes neurais recorrentes (via `TensorFlow` ou `PyTorch`) podem ser testados para potencialmente melhorar a capacidade preditiva em datasets textuais. As redes neurais são o assunto da Parte II.

2. **Expansão do conjunto de dados**: Aumentar o número de exemplos rotulados, seja por anotação manual ou por integração com fontes externas (ex.: APIs de notícias), pode aprimorar a generalização do modelo.

3. **Otimização de hiperparâmetros**: Ajustes nos parâmetros do SVM, como a escolha de kernels alternativos (`kernel='rbf'`) ou a variação do parâmetro de regularização (`C`), podem ser explorados via validação cruzada para otimizar o desempenho.

4. **Classificação em tempo real**: Implementar um sistema que colete notícias em tempo real (ex.: utilizando `requests` e `BeautifulSoup`) e aplique o modelo treinado pode transformar o projeto em uma ferramenta dinâmica.

5. **Extensão multilíngue**: Adaptar o pipeline para processar textos em outros idiomas, ajustando o pré-processamento (ex.: `stop_words='english'`) e utilizando datasets multilíngues, amplia o alcance da análise de sentimentos.

Parte II

Deep Learning

Capítulo 8

Do Neurônio às Redes Neurais

A regressão logística do Capítulo 4 já é um neurônio artificial: ela multiplica cada entrada por um peso, soma os resultados e passa essa soma pela função sigmoide. Uma rede neural empilha várias dessas unidades em camadas, e esse empilhamento permite aprender fronteiras de decisão que uma única reta não consegue traçar. A Parte II começa nesse ponto e segue até o treinamento de redes com várias camadas.

Capítulo em desenvolvimento. Tópicos previstos:

- O neurônio artificial e o Perceptron.
- Da regressão logística ao Perceptron Multicamadas (MLP).
- Funções de ativação: sigmoide, ReLU e softmax.
- Propagação direta (*forward pass*) calculada à mão em uma rede pequena.

Capítulo 9

Como uma Rede Aprende

Capítulo em desenvolvimento. Tópicos previstos:

- O aprendizado como problema de otimização: funções de perda.
- Derivadas, derivadas parciais e o vetor gradiente.
- Gradiente descendente e taxa de aprendizado.
- Regra da cadeia e retropropagação (*backpropagation*).
- Uma iteração de treino resolvida à mão e conferida em Python.

Capítulo 10

Treinando Redes Neurais na Prática

Capítulo em desenvolvimento. Tópicos previstos:

- Implementação de uma rede em Python.
- Épocas, lotes (*batches*) e curvas de perda.
- *Overfitting* em redes: regularização, *dropout* e parada antecipada.
- Projeto prático da Parte II.

Capítulo 11

Recursos e Considerações Finais

11.1 Recursos Recomendados

Os seguintes recursos são indicados para aprofundamento no estudo de aprendizado de máquina e análise de sentimentos:

- **Livros**: *Python Machine Learning* de Sebastian Raschka e Vahid Mirjalili oferece uma base sólida em algoritmos de aprendizado de máquina, enquanto *Deep Learning with Python* de François Chollet introduz redes neurais de forma prática. E não poderia faltar *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, de Aurélien Géron, que traz extenso conteúdo sobre o assunto.

- **Cursos**: O curso *Machine Learning* de Andrew Ng na Coursera é uma referência em fundamentos teóricos, e o curso *Data Science com Python* da DataCamp foca em aplicações práticas.

- **Documentação**: A documentação oficial do `scikit-learn` (https://scikit-learn.org) detalha as funcionalidades de pré-processamento e modelagem usadas neste livro.

- **Comunidades**: Fóruns como o subreddit *r/MachineLearning* (https://reddit.com/r/MachineLearning) e comunidades de ciência de dados no Discord são espaços para discussão e troca de experiências.

- **Conjuntos de dados**: Plataformas como Kaggle (https://kaggle.com) e o UCI Machine Learning Repository (https://archive.ics.uci.edu) disponibilizam datasets variados para experimentação.

11.2 Considerações Finais

A Parte I deste livro apresentou os fundamentos do aprendizado de máquina clássico, da estatística básica à regressão, à classificação e ao clustering, e terminou com um pro-

jeto completo de análise de sentimentos. As etapas percorridas (aquisição de dados, pré-processamento de texto, treinamento de modelos e avaliação) formam a base sobre a qual as partes de Deep Learning e Visão Computacional se apoiam. Encorajamos os leitores a explorar as sugestões avançadas e os recursos recomendados, aplicando os conceitos aprendidos em novos contextos e contribuindo para o avanço de suas habilidades em ciência de dados.

Referências Bibliográficas

[1] Bruce, P., & Bruce, A. (2017). *Estatística prática para cientistas de dados*. O'Reilly Media.

[2] Knaflic, C. N. (2015). *Storytelling com dados: um guia sobre visualização de dados para profissionais de negócios*. Wiley.

[3] Géron, A. (2023). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems* (3rd ed.). O'Reilly Media.

[4] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

[5] Scikit-learn developers. (2024). *Scikit-learn: Machine Learning in Python*. Disponível em https://scikit-learn.org. Acesso em: 1 abr. 2025.

[6] Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. O'Reilly Media.

[7] Bussab, W. de O., & Morettin, P. A. (2017). *Estatística Básica* (9ª ed.). Saraiva Educação.